

DEPARTMENT OF COMPUTER ENGINEERING

LABORATORY MANUAL

MACHINE LEARNING LABORATORY

(BTCOL606B)



**Shri Vile Parle Kelavani Mandal's
INSTITUTE OF TECHNOLOGY DHULE (M.S.)**

DEPARTMENT OF COMPUTER ENGINEERING

Vision of Institute

To be a socially sensitive Institute of Excellence adding value to the nation.

Mission of Institute

1. To provide resources of excellence with a focus on nurturing and developing the society.
2. To strive to be an institute of global recognition.

Vision of Department

We envision a globally recognized and innovative computer engineer who meets socio-economical and industrial needs.

Mission of Department

1. To empower students with comprehensive knowledge of Computer Engineering to be successful professionals.
2. To ensure quality education that prepares students for careers in industry and higher education.
3. To develop leadership skills in students while instilling strong ethical values and encouraging lifelong learning.

Program Educational Objectives (PEOs) of Department

PEO I: Graduates will have a successful professional career while maintaining strong ethical values.

PEO II: Graduates must demonstrate knowledge and technological skills to resolve real life problems.

PEO III: Graduates will progress as professionals, researchers, or entrepreneurs who continues to learn and adopt emerging technology.

COMPUTER ENGINEERING DEPARTMENT

Program Outcomes

Engineering Graduates will be able to:

- 1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- 2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- 3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- 4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- 5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- 6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- 7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- 8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- 9. Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- 10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- 11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- 12. Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COMPUTER ENGINEERING DEPARTMENT

Program Specific Outcomes (PSO) addressed by the Course:

A graduate of the Computer Engineering Program will demonstrate-

PSO1: Professional Skills-To gain the ability to comprehend, analyze, design and implement computer programs in the fields of computer algorithms, web development, data science, computer network and security, software design, system software cloud computing and allied fields.

PSO2: Problem-Solving Skills- Capability to provide computer based solutions to a variety of problems by applying standard practices, problem solving strategies and methodologies.

PSO3: Professional Career - The ability to create an innovative career path by utilizing modern computer tools and technologies.

Dr. Babasaheb Ambedkar Technological University

Syllabus & Scheme

Course Code	Course Title	Weekly Teaching hrs	Evaluation Scheme			Credit
			MSE	CA	ESE	
BTCOL606B	Machine Learning Laboratory	2	-	40	60	1

BTCOL508 Machine Learning Laboratory

As a part of lab exercises for Machine Learning Laboratory, it is suggested that the student should get hands-on experience by solving data analysis problems available on Machine Learning competition platforms such as Hacker Earth and Kaggle. Some of the suggestive list of problem solving is given below. Knowledge of R programming or Python is required to solve these problems, students get this prerequisite in Second Year.

1 Regression Analysis and Plot interpretation.

2 Logistic Regression Analysis in R.

3 Random Forest and Parameter Tuning in R.

4 Clustering Algorithms and Evaluation in R.

5 Machine Learning Project in Python on House Prices Data.



Shri Vile Parle Kelavani Mandal's
Institute of Technology, Dhule.

www.svkm-iot.ac.in

Survey No. 499, Plot No.2, Behind Gurudwara, Mumbai-Agra Road, Dhule(M.S.)

DEPARTMENT OF COMPUTER ENGG.

LABORATORY : BTCOL508 Machine Learning Laboratory

SEMESTER : 5th Sem

Master List of Practical

SR. NO.	PRACTICAL DESCRIPTION	Page No
1	Write a program to Perform Exploratory Data Analysis and Data Visualization in Python.	11-15
2	Write a program to Perform Data Preprocessing through various feature-engineering algorithms by python.	16-24
3	Write a program to implement the Implement Simple Linear Regression and Multiple Linear Regression for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.	25-31
4	Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample and Visualize model with Python.	32-38
5	Write a program to implement k-Nearest Neighbor algorithm to classify the iris data set. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.	39-44
6	Write a program to implement the Kernel SVM Algorithm for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.	45-50
7	Write a program to implement the Random Forest Algorithm for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.	51-58
8	Write a program to implement the K-means Clustering Algorithm for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.	59-62
9	Write a program to implement the Hierarchical Agglomerative Clustering Algorithm for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.	63-68
10	Mini Project : Create the Model Evaluation for a real life training data set stored as a .CSV file and by computing accuracy for different model proposed the best suitable machine learning algorithm for problem or Solve the any real life problem suggested by Hackerearth site	69-76

Subject In-charge:
 Prof. Ashish Awate

APPROVED BY :
 Dr. Makarand Shahade
 HOD, Department of Computer Engineering

Rubrics for Assessment

Assignment/Experiment :

Date of Performance:

Date of submission :

Marks Split Up to	Maximum Marks	Marks Obtained
Performance/ conduction	3	
Report Writing	3	
Attendance	2	
Viva/Oral	2	
Total Marks	10	
Signature of Subject Teacher		

COURSE OBJECTIVES:

The objective of the course is

1. To understand the basic theory underlying machine learning.
2. To be able to formulate machine learning problems corresponding to different applications.
3. To understand a range of machine learning algorithms along with their strengths and weaknesses.
4. To be able to apply machine learning algorithms to solve problems of moderate complexity.
5. To apply the algorithms to a real-world problem, optimize the models learned and report on the expected accuracy that can be achieved by applying the models.

COURSE OUTCOMES:

CO508.1: Understand the mathematical and statistical prospective of machine learning algorithms through python programming.

CO508.2: Evaluate the machine learning models pre-processed through various feature engineering algorithms by python programming.

CO508.3: Design and evaluate the supervised models through python in built functions.

CO508.4: Design and evaluate the unsupervised models through python in built functions.

CO508.5: Recognize and implement various ways of selecting suitable model parameters for different machine learning techniques.

GUIDELINES FOR STUDENT'S LAB JOURNAL

1. The laboratory assignments are to be submitted by student in the form of journal.
2. The Journal consists of prologue, Certificate, table of contents, and handwritten write-up of each assignment (Title, Objectives, Problem Statement, Outcomes, software & Hardware requirements, Date of Completion, Assessment grade/marks and assessor's sign, TheoryConcept, circuit diagram, pin configuration, conclusion/analysis), printouts of the code written using coding standards, sample test cases etc.
3. Practical Examination will be based on the term work submitted by the student in the form of Journal
4. Candidate is expected to know the theory involved in the experiment
5. The practical examination should be conducted if the journal of the candidate is completed in all respects and certified by concerned faculty and head of the department
6. All the assignment mentioned in the syllabus must be conducted.

Experiment No. 1

Title : Exploratory Data Analysis and Data Visualization in Python

Problem Statement : The main aim of this lab is to demonstrate the methods used or data analysis and visualization from the Pandas, Seaborn and Matplotlib library of Python.

The secondary objectives are:

1. to explain how a dataset is loaded as Pandas data frame,
2. how the required information is extracted from the data frame, and
3. how the dataset is visualized through different kinds of graphs.

Software Required : Jupyter Notebook , Python 3.6, Or Google Colab

Dataset : Titanic.csv

Procedure and Activity:

Dataset initialization: The dataset initialization can be divided into two parts. First is importing necessary modules. Here, we import Pandas, Matplotlib and Seaborn modules.

The second activity in the data initialization includes loading of the dataset into memory for processing. Here, we are assuming a structured and cross-sectional dataset. The structured dataset is usually available in CSV (Comma Separated Variables) format. By cross-sectional, I mean data instances are collected over the period of a single duration.

Python's Pandas library provides the method `read_csv()` to read a CSV file and to load it as a data frame. The data frame is two-dimensional data structures like the matrix. It can be indexed through non-integer variables also. Further, the data frame method `head()` displays complete information of the first five instances in the dataset which is also referred to as the header

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

```
df=pd.read_csv('../input/titanicdataset-traincsv/train.csv')
df.head()
```

The next activity is to get the meta-information about the dataset. The attribute `shape` and `columns` provide information about the dimensional size of the dataset and names of features or dataset variables.

The method `info()` provides information about feature name, feature type, and whether there are any missing values are present for any of the features.

```
print(df.shape)
```

```
df.columns
```

```
print(df.info())
```

The descriptive statistics of numeric variables is obtained through the data frame method `describe()`. For each numeric data feature, this method displays the total number of instances (count), mean, standard deviation (std), minimum value (min), first quartile (25%), third quartile (75%) and the maximum value.

This method is quite useful to get the initial insight into the dataset.

In case the parameter `'include="object"'` is specified, the method `describe()` displays the descriptive statistics of categorical or discrete type of variables. For categorical variables, this method shows information such as

a specific number of values for a given categorical variable, the value occurring most of the times in the data set, and its count.

```
df.describe()
```

```
df.describe(include='object')
```

The method **value_counts()** which is invoked on a specific column of a data frame is another useful method to count the occurrences of individual values for a given feature variable. The parameter 'normalize=True' can be used to get a percentage of occurrences of individual value in a specified column.

```
df['Sex'].value_counts()
```

```
df['Sex'].value_counts(normalize=True)
```

Summary Table 1

Task	Command
To load a CSV file	<code>df= pd.read_csv()</code>
To Print the header of CSV file	<code>df.head()</code>
To print dimensional size of the data set	<code>df.shape()</code>
To print names of variables or features	<code>df.columns()</code>
To print types of variables or features	<code>df.info()</code>
To print descriptive statistics of numeric variables	<code>df.describe()</code>
To print descriptive statistics of non-numeric variables	<code>df.describe(include= 'object')</code>
To print counts of distinct values for categorical variables	<code>df['Sex'].value_counts()</code>
To print percentage of distinct values for categorical variables	<code>df['Sex'].value_counts(normalize=True)</code>

Querying a dataset or DataFrame

- Querying a dataset or DataFrame with Pandas Pandas provides several methods to query a particular dataframe.
- The query methods provided in Pandas are as expressive as statements in any query languages (e.g., SQL) as supported by relational databases.
- This section illustrates the application of query mechanisms for the given Titanic dataset. Projecting columns or feature variables
- There are multiple ways to access a particular column or feature variable in Pandas.
- The first and simplest way is to index the dataframe with the name of the desired column.
- For example, `*df['Name']*` returns all the values in the 'Name' column as a Pandas's series.
- Further, we can invoke the methods such as `head()`, `mean()` and `max()` on the column values returned as series.

```
df['Name'].head()
```

The second way of projecting columns and instances is to use `loc[]` and `iloc[]` methods. The `loc[]` method accesses columns with column name as indexes. At the same time, `iloc[]` uses integers as an index which indicate the column position—use of square brackets[] to be noted here.

```
df.loc[0:15, 'Name': 'Age']
```

```
df.iloc[0:15, 3:6]
```

More complex queries can be constructed to extract rows or data instances, satisfying a particular condition—for example, printing name of the oldest passenger. Here, we need to first find the largest values for the age and print the data instances satisfying the condition `age == max age`.

```
df[df['Age']== df[df['Sex'] =='male']['Age'].max()]['Name']
```

Sorting Values in a Column: The method `sort_values()` can be used to sort the values in a dataframe's column. It takes two arguments. The first is the name of a column to be sorted (by) and the second is the order of sorting. (ascending). Here are a few examples.

```
df.sort_values(by='Name').head()
```

```
df.sort_values(by=['Name', 'Age'], ascending=[True, False]).head()
```

Replacing values in a column:

Sometimes, we need to replace all values in a column. Such situations arise when we want to convert all categorical values into supposing numeric values. Pandas provides two convenient ways to achieve this task. The first is through the application of `map()` method and the second is through the `replace()` method.

The following example replaces the values 'male' and 'female' in the 'Sex' column with values '0' and '1'. Here we define a new dictionary with old values of the column as keys (male, female) and new values as the value(0,1) in the dictionary. Then this dictionary is passed as an argument to the `map()` method.

```
d={'male':0, 'female':1}
df['Sex'] = df['Sex'].map(d)
df.head()
```

The second way of replacing values in a column is through the method `replace()`. Like in the previous example, we need to define a dictionary. The newly created dictionary is included as a value and column name as the key in the dictionary to be passed as an argument to the `replace()` method.

The following example restores the values in the 'Sex' column back to 'male' and 'female' through `replace()` method.

```
o={0: 'male', 1: 'female'}
df= df.replace({'Sex':o})
df.head()
```

Grouping columns value-wise:

The method `groupby()` can be used to group the data instances according to the values of a categorical variable and print the descriptive statistics value-wise.

For example, the data instances in the Titanic data set are grouped for 'Survived' (a categorical variable) and an average 'Age' (a numerical variable) for survived and not survived passengers are calculated.

```
df.groupby(by='Survived')['Age'].describe()
```

Another way of grouping data instances or rows is through `crosstab()` method. It allows to group rows through multiple categorical variables.

The following example groups rows by 'Sex' and 'Survived' columns and presents genderwise percentage (normalize=True) of survived and not survived passengers.

```
pd.crosstab(df['Sex'],df['Survived'], normalize=True)
```

The method `pivot_table()` is the more flexible way of grouping rows. It allows to group rows through a categorical variable and presents the descriptive statistics for multiple numerical variables.

```
df.pivot_table(['Age', 'Fare'], ['Survived'], aggfunc='mean')
```

Summary Table 2

Task	Method
Projecting a column i.e. a feature	<code>df.['fname']</code>
Selecting rows and columns	<code>Df.loc[0:5, 'fname':fname]</code> <code>df.iloc[0:5, 0:5]</code>
Sorting	<code>df.sort values(by=[], ascending=[])</code>
Replacing values in a column	Map and replace method
Grouping values	<code>df.groupby() [].describe()</code>
Relating two or more than two variables	<code>df.crosstab()</code> <code>df.pivottable()</code>

Data Visualization

Python provides diverse ways to visualize dataset through libraries such as Pandas, Matplotlib and Seaborn.

The objective of this section is to familiarize with different methods of visualization and how different kinds of graphs such as linechart, barchart, histogram, and scatter plot are plotted.

Line Graph : The first and most straightforward way is to depict a data set through a line graph. The line graph is typically used for numerical and time-series data.

Here we use the *lineplot()* method from Seaborn to plot a graph age vs fare paid. Both are numeric data. In the line plot, two points are joined through lines.

```
sns.lineplot(x='Age', y='Fare', data=df)
```

Bar Graph : Python provides diverse ways to visualize the dataset. The second kind of graph that is frequently used to depict the data is the bar graph. The bar graphs are used to display counts of values for categorical or a discrete variable.

Here we use a bar graph to display counts of survived and not survived passengers as a bar graph through the *factorplot()* method from Seaborn library.

```
sns.factorplot(x='Survived', data=df, kind='count')
```

The *factorplot()* method can take an additional categorical variable through the 'hue' parameter. Here in the following example, we display the genderwise distribution of survived and not-survived passengers.

```
sns.factorplot(x='Survived', data=df, kind='count', hue='Sex')
```

Histograms : The histograms can be drawn through the *hist()* method from Pandas library. The *hist()* method takes the number of bins as one of the arguments. The histograms are useful to visualize the distribution of numeric of data. Here in this example, we display distribution of age through histograms.

From the histograms, it can be easily inferred that the majority of the passengers belong to age group 20-40years.

```
df['Age'].hist(bins=8)
```

Scatter plot : The scatter plot is used to observe how one variable relates to another variable. The `scatter()` method from Pandas library is used to draw the scatter plot.

The following examples plot two scatter plots. First is 'Age' vs 'Fare' and the second 'Age' vs 'Survived'.

```
df.plot.scatter(x='Age',y='Fare')  
  
df.plot.scatter(x='Age',y='Survived')
```

Pie Chart : Plotting a pie chart is somewhat complicated in Python. Here we use the method `subplots()` from matplotlib and draw a pie chart to display value-wise composition for the 'Sex' variable.

```
import matplotlib.pyplot as plt  
sizes= df['Survived'].value_counts()  
fig1,ax1 = plt.subplots()  
ax1.pie(sizes,labels=['Not Survived', 'Survived'],autopct='%1.1f%%',shadow=True)  
plt.show()
```

Task to Perform: Download the dataset and perform above EDA Steps and Data Visualization for it.

Conclusion : In this way we successfully completed the Exploratory data analysis and visualization from the Pandas, Seaborn and Matplotlib library of Python.

Experiment No. 2

Title : Write a program to Perform Data Preprocessing through various feature-engineering algorithms by python.

Problem Statement : The main aim of this lab is to demonstrate the data preprocessing techniques used in machine learning using Python

The secondary objectives are:

4. to explain how to deal with missing data,
5. how to normalize data
6. how to split data into train and test dataset
7. how to deal with categorical data.
8. Rescaling of data

Software Required : Jupyter Notebook , Python 3.6, Or Google Colab

Dataset : Data.csv

Theory :

Data Preprocessing : Concepts

Data is truly considered a resource in today's world. As per the World Economic Forum, by 2025 we will be generating about 463 exabytes of data globally per day! But is all this data fit enough to be used by machine learning algorithms? How do we decide that? In this article we will explore the topic of data preprocessing — transforming the data such that it becomes machine-readable...

What is Data Preprocessing?

In any Machine Learning process, Data Preprocessing is that step in which the data gets transformed, or Encoded, to bring it to such a state that now the machine can easily parse it. In other words, the features of the data can now be easily interpreted by the algorithm.

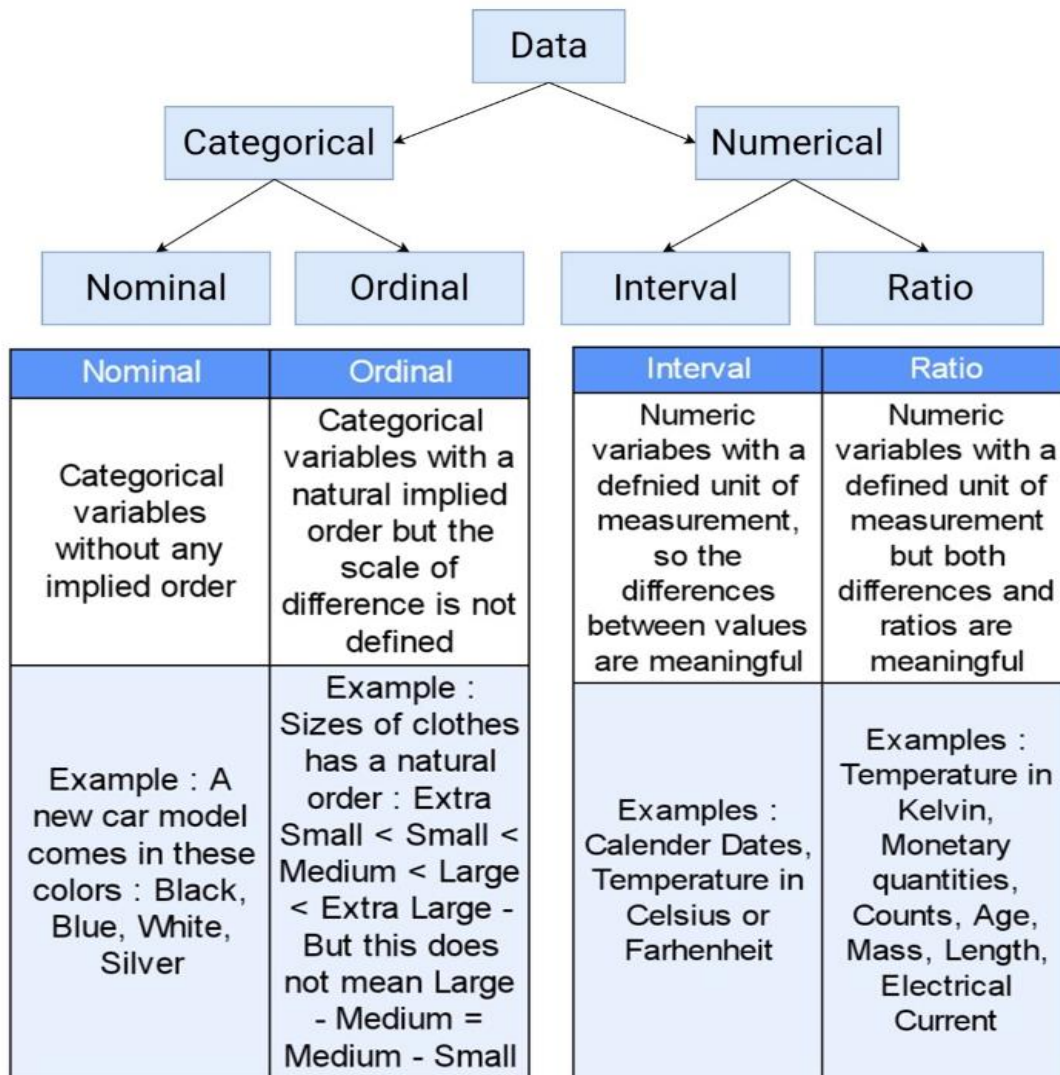
Features in Machine Learning

A dataset can be viewed as a collection of data objects, which are often also called as a records, points, vectors, patterns, events, cases, samples, observations, or entities.

Data objects are described by a number of features, that capture the basic characteristics of an object, such as the mass of a physical object or the time at which an event occurred, etc.. Features are often called as variables, characteristics, fields, attributes, or dimensions.

“A feature is an individual measurable property or characteristic of a phenomenon being observed”

For instance, color, mileage and power can be considered as features of a car. There are different types of features that we can come across when we deal with data.



Generally they are :

- Data Quality Assessment
- Feature Aggregation
- Feature Sampling
- Dimensionality Reduction
- Feature Encoding

Data Quality Assessment

Because data is often taken from multiple sources which are normally not too reliable and that too in different formats, more than half our time is consumed in dealing with data quality issues when working on a machine learning problem. It is simply unrealistic to expect that the data will be perfect. There may be problems due to human error, limitations of measuring devices, or flaws in the data collection process. Let's go over a few of them and methods to deal with them :

1. Missing values :

It is very much usual to have missing values in your dataset. It may have happened during data collection, or maybe due to some data validation rule, but regardless missing values must be taken into consideration.

a. Eliminate rows with missing data :

Simple and sometimes effective strategy. Fails if many objects have missing values. If a feature has mostly missing values, then that feature itself can also be eliminated.

b. Estimate missing values :

If only a reasonable percentage of values are missing, then we can also run simple interpolation methods to fill in those values. However, most common method of dealing with missing values is by filling them in with the mean, median or mode value of the respective feature.

2. Inconsistent values :

We know that data can contain inconsistent values. Most probably we have already faced this issue at some point. For instance, the 'Address' field contains the 'Phone number'. It may be due to human error or maybe the information was misread while being scanned from a handwritten form.

It is therefore always advised to perform data assessment like knowing what the data type of the features should be and whether it is the same for all the data objects.

3. Duplicate values :

A dataset may include data objects which are duplicates of one another. It may happen when say the same person submits a form more than once. The term deduplication is often used to refer to the process of dealing with duplicates.

In most cases, the duplicates are removed so as to not give that particular data object an advantage or bias, when running machine learning algorithms.

Feature Aggregation

Feature Aggregations are performed so as to take the aggregated values in order to put the data in a better perspective. Think of transactional data, suppose we have day-to-day transactions of a product from recording the daily sales of that product in various store locations over the year. Aggregating the transactions to single store-wide monthly or yearly transactions will help us reducing hundreds or potentially thousands of transactions that occur daily at a specific store, thereby reducing the number of data objects.

This results in reduction of memory consumption and processing time

Aggregations provide us with a high-level view of the data as the behavior of groups or aggregates is more stable than individual data objects

Feature Sampling

Sampling is a very common method for selecting a subset of the dataset that we are analyzing. In most cases, working with the complete dataset can turn out to be too expensive considering the memory and time constraints. Using a sampling algorithm can help us reduce the size of the dataset to a point where we can use a better, but more expensive, machine learning algorithm.

The key principle here is that the sampling should be done in such a manner that the sample generated should have approximately the same properties as the original dataset, meaning that the sample is representative. This involves choosing the correct sample size and sampling strategy.

Simple Random Sampling dictates that there is an equal probability of selecting any particular entity. It has two main variations as well :

- **Sampling without Replacement** : As each item is selected, it is removed from the set of all the objects that form the total dataset.
- **Sampling with Replacement** : Items are not removed from the total dataset after getting selected. This means they can get selected more than once.

Although Simple Random Sampling provides two great sampling techniques, it can fail to output a representative sample when the dataset includes object types which vary drastically in ratio. This can cause problems when the sample needs to have a proper representation of all object types, for example, when we have an imbalanced dataset.

Feature Encoding

As mentioned before, the whole purpose of data preprocessing is to *encode* the data in order to bring it to such a state that the machine now understands it.

Feature encoding is basically performing transformations on the data such that it can be easily accepted as input for machine learning algorithms while still retaining its original meaning.

There are some general norms or rules which are followed when performing feature encoding. For Continuous variables :

- **Nominal** : Any one-to-one mapping can be done which retains the meaning. For instance, a permutation of values like in One-Hot Encoding.

	Name	Generation	Gen 1	Gen 2	Gen 3	Gen 4	Gen 5
4	Octillery	Gen 2	0	1	0	0	0
5	Helioptile	Gen 6	0	0	0	0	0
6	Dialga	Gen 4	0	0	0	1	0
7	DeoxysDefense Forme	Gen 3	0	0	1	0	0
8	Rapidash	Gen 1	1	0	0	0	0
9	Swanna	Gen 5	0	0	0	0	1

- **Ordinal** : An order-preserving change of values. The notion of small, medium and large can be represented equally well with the help of a new function, that is, $\langle \text{new_value} = f(\text{old_value}) \rangle$ - For example, {0, 1, 2} or maybe {1, 2, 3}.

Numeric variables:

- **Interval** : Simple mathematical transformation like using the equation $\langle \text{new_value} = a * \text{old_value} + b \rangle$, a and b being constants. For example, Fahrenheit and Celsius scales, which differ in their Zero values size of a unit, can be encoded in this manner.
- **Ratio** : These variables can be scaled to any particular measures, of course while still maintaining the meaning and ratio of their values. Simple mathematical transformations work in this case as well, like the transformation $\langle \text{new_value} = a * \text{old_value} \rangle$. For, length can be measured in meters or feet, money can be taken in different currencies.

Train / Validation / Test Split

After feature encoding is done, our dataset is ready for the exciting machine learning algorithms! But before we start deciding the algorithm which should be used, it is always advised to split the dataset into 2 or sometimes 3 parts. Machine Learning algorithms, or any algorithm for that matter, has to be first trained on the data distribution available and then validated and tested, before it can be deployed to deal with real-world data.

Training data : This is the part on which your machine learning algorithms are actually trained to build a model. The model tries to *learn* the dataset and its various characteristics and intricacies, which also raises the issue of Overfitting v/s Underfitting.

Validation data : This is the part of the dataset which is used to validate our various model fits. In simpler words, we use validation data to choose and improve our model hyperparameters. The model does not *learn* the validation set but uses it to get to a better state of hyperparameters.

Test data : This part of the dataset is used to test our model hypothesis. It is left untouched and unseen until the model and hyperparameters are decided, and only after that the model is applied on the test data to get an accurate measure of how it would perform when deployed on real-world data.



Split Ratio :

Data is split as per a split ratio which is highly dependent on the type of model we are building and the dataset itself. If our dataset and model are such that a lot of training is required, then we use a larger chunk of the data just for training purposes (usually the case) — For instance, training on textual data, image data, or video data usually involves thousands of features!

If the model has a lot of hyperparameters that can be tuned, then keeping a higher percentage of data for the validation set is advisable. Models with less number of hyperparameters are easy to tune and update, and so we can keep a smaller validation set.

Like many other things in Machine Learning, the split ratio is highly dependent on the problem we are trying to solve and must be decided after taking into account all the various details about the model and the dataset in hand.

Procedure: Steps in Data Preprocessing in Machine Learning

There are seven significant steps in data preprocessing in Machine Learning:

1. Acquire the dataset

To build and develop Machine Learning models, you must first acquire the relevant dataset. This dataset will be comprised of data gathered from multiple and disparate sources which are then combined in a proper format to form a dataset. Dataset formats differ according to use cases. For instance, a business dataset will be entirely different from a medical dataset. While a business dataset will contain relevant industry and business data, a medical dataset will include healthcare-related data.

There are several online sources from where you can download datasets like <https://www.kaggle.com/uciml/datasets> and <https://archive.ics.uci.edu/ml/index.php>. You can also create a dataset by collecting data via different Python APIs. Once the dataset is ready, you must put it in a CSV, or HTML, or XLSX file formats.

2. Import all the crucial libraries

Since Python is the most extensively used and also the most preferred library by Data Scientists around the world, we'll show you how to import Python libraries for data preprocessing in Machine Learning. Read more about [Python libraries for Data Science here](#). The predefined Python libraries can perform specific data preprocessing jobs. The three core Python libraries used for this data preprocessing in Machine Learning are:

- **NumPy** – NumPy is the fundamental package for scientific calculation in Python. Hence, it is used for inserting any type of mathematical operation in the code. Using NumPy, you can also add large multidimensional arrays and matrices in your code.
- **Pandas** – Pandas is an excellent open-source Python library for data manipulation and analysis. It is extensively used for importing and managing the datasets. It packs in high-performance, easy-to-use data structures and data analysis tools for Python.

- **Matplotlib** – Matplotlib is a Python 2D plotting library that is used to plot any type of charts in Python. It can deliver publication-quality figures in numerous hard copy formats and interactive environments across platforms (IPython shells, Jupyter notebook, web application servers, etc.).

3. Import the dataset

In this step, you need to import the dataset/s that you have gathered for the ML project at hand. However, before you can import the dataset/s, you must set the current directory as the working directory. You can set the working directory in Spyder IDE in three simple steps:

- I. Save your Python file in the directory containing the dataset.
- II. Go to File Explorer option in Spyder IDE and choose the required directory.
- III. Now, click on the F5 button or Run option to execute the file.

Once you've set the working directory containing the relevant dataset, you can import the dataset using the “read_csv()” function of the Pandas library. This function can read a CSV file (either locally or through a URL) and also perform various operations on it. The read_csv() is written as:

```
data_set= pd.read_csv('Dataset_name.csv/path/url')
```

In this line of code, “data_set” denotes the name of the variable wherein you stored the dataset. The function contains the name of the dataset as well. Once you execute this code, the dataset will be successfully imported.

During the dataset importing process, there's another essential thing you must do – extracting dependent and independent variables. For every Machine Learning model, it is necessary to separate the independent variables (matrix of features) and dependent variables in a dataset.

How to extract the independent variables?

To extract the independent variables, you can use “iloc[]” function of the Pandas library. This function can extract selected rows and columns from the dataset.

```
x= data_set.iloc[:, :-1].values
```

In the line of code above, the first colon(:) considers all the rows and the second colon(:) considers all the columns. The code contains “:-1” since you have to leave out the last column containing the dependent variable.

How to extract the dependent variable?

You can use the “iloc[]” function to extract the dependent variable as well. Here's how you write it:

```
y= data_set.iloc[:, -1].values
```

This line of code considers all the rows with the last column only.

4. Identifying and handling the missing values

In data preprocessing, it is pivotal to identify and correctly handle the missing values, failing to do this, you might draw inaccurate and faulty conclusions and inferences from the data. Needless to say, this will hamper your ML project.

Basically, there are two ways to handle missing data:

Deleting a particular row – In this method, you remove a specific row that has a null value for a feature or a particular column where **more than 75%** of the values are missing. However, this **method is not 100% efficient**, and it is recommended that you use it only when the dataset has adequate samples. You must ensure that after deleting the data, there remains no addition of bias.

Calculating the mean – This method is useful for features having numeric data like age, salary, year, etc. Here, you can calculate the mean, median, or mode of a particular feature or column or row that contains a missing value and replace the result for the missing value. This method can add variance to the dataset, and any loss of data can be efficiently negated. Hence, it yields better results compared to the first method (omission of rows/columns). Another way of approximation is through the deviation of neighbouring values. However, this works best for linear data.

5. Encoding the categorical data

Categorical data refers to the information that has specific categories within the dataset.

The labelEncoder and OneHotEncoder only works on categorical features.

Label Encoding refers to converting the labels into numeric form so as to convert it into the machine-readable form. Machine learning algorithms can then decide in a better way on how those labels must be operated. It is an important pre-processing step for the structured dataset in supervised learning.

We need first to extract the categorial featuers using boolean mask.

```
# Categorical boolean mask

categorical_feature_mask = X.dtypes==object# filter categorical columns using mask
and turn it into a list

categorical_cols = X.columns[categorical_feature_mask].tolist()
```

LabelEncoder converts each class under specified feature to a numerical value. Let's go through the steps to see how to do it.

Instantiate a LabelEncoder object:

```
# import labelencoder

from sklearn.preprocessing import LabelEncoder# instantiate labelencoder object

le = LabelEncoder()
```

Apply LabelEncoder on each of the categorical columns:

```
# apply le on categorical feature columns

X[categorical_cols] = X[categorical_cols].apply(lambda col:
le.fit_transform(col))X[categorical_cols].head(10)
```

Limitation of label Encoding

Label encoding convert the data in machine readable form, but it assigns a unique number(starting from 0) to each class of data. This may lead to the generation of priority issue in training of data sets. A label with high value may be considered to have high priority than a label having lower value.

Thus allowing model learning this result will lead to poor performance. Therefore, for dataframe containing multi class features, a further step of OneHotEncoder is needed. Let's see the steps to do it.

Instantiate OneHotEncoder object:

```
# import OneHotEncoder

from sklearn.preprocessing import OneHotEncoder# instantiate OneHotEncoder

ohe = OneHotEncoder(categorical_features = categorical_feature_mask, sparse=False )

# categorical_features = boolean mask for categorical columns

# sparse = False output an array not sparse matrix
```

We need to specify categorical feature using its mask inside OneHotEncoder. The `sparse=False` argument outputs a non-sparse matrix.

Apply OneHotEncoder on DataFrame:

```
# apply OneHotEncoder on categorical feature columns

X_ohe = ohe.fit_transform(X) # It returns an numpy array
```

Note that the output is a numpy array, not a dataframe. For each class under a categorical feature, a new column is created for it.

6. Splitting the dataset

Every dataset for Machine Learning model must be split into two separate sets – training set and test set.

Training set denotes the subset of a dataset that is used for training the machine learning model. Here, you are already aware of the output. A test set, on the other hand, is the subset of the dataset that is used for testing the machine learning model. The ML model uses the test set to predict outcomes.

Usually, the dataset is split into 70:30 ratio or 80:20 ratio. This means that you either take 70% or 80% of the data for training the model while leaving out the rest 30% or 20%. The splitting process varies according to the shape and size of the dataset in question.

To split the dataset, you have to write the following line of code –

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.2,
random_state=0)
```

Here, the first line splits the arrays of the dataset into random train and test subsets. The second line of code includes four variables:

- `x_train` – features for the training data
- `x_test` – features for the test data
- `y_train` – dependent variables for training data
- `y_test` – independent variable for testing data

Thus, the `train_test_split()` function includes four parameters, the first two of which are for arrays of data. The `test_size` function specifies the size of the test set. The `test_size` maybe .5, .3, or .2 – this specifies the dividing ratio between the training and test sets. The last parameter, “`random_state`” sets seed for a random generator so that the output is always the same.

7. Feature scaling

Feature scaling marks the end of the **data preprocessing in Machine Learning**. It is a method to standardize the independent variables of a dataset within a specific range. In other words, feature scaling limits the range of variables so that you can compare them on common grounds. Thus, you must remove this issue by performing feature scaling for Machine Learning.

Most ML models are based on Euclidean Distance, which is represented as:

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

You can perform feature scaling in Machine Learning in two ways:

1. **Standardize Data:** Standardization is a useful technique to transform attributes with a Gaussian distribution and differing means and standard deviations to a standard Gaussian distribution with a mean of 0 and a standard deviation of 1. We can standardize data using scikit-learn with the StandardScaler class

$$X' = \frac{x - \text{mean}(x)}{a}$$

2. Rescale Data

- When our data is comprised of attributes with varying scales, many machine learning algorithms can benefit from rescaling the attributes to all have the same scale.
- This is useful for optimization algorithms in used in the core of machine learning algorithms like gradient descent.
- It is also useful for algorithms that weight inputs like regression and neural networks and algorithms that use distance measures like K-Nearest Neighbors.
- We can rescale your data using scikit-learn using the **MinMaxScaler** class.

$$X' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Task to Perform: Download the dataset and perform above 7 steps for data preprocessing on it

Conclusion : In this way successfully implemented data preprocessing technique on our dataset

Experiment No. 3

Title : Write a program to implement the Implement Simple Linear Regression and Multiple Linear Regression for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.

Problem Statement : The main aim of this lab is to demonstrate the how to implement the simple linear regression and multiple linear regression Python.

Secondary aim of this lab is to undersand

- What linear regression is
- What linear regression is used for
- How linear regression works
- How to implement linear regression in Python, step by step

Software Used: Jupyter Notebook , Python 3.6, Or Google Colab

Dataset : Salary_Data.csv, 50_Startups.csv

Theory:

Regression

Regression analysis is one of the most important fields in statistics and machine learning. There are many regression methods available. Linear regression is one of them.

What Is Regression?

Regression searches for relationships among variables.

For example, you can observe several employees of some company and try to understand how their salaries depend on the **features**, such as experience, level of education, role, city they work in, and so on.

This is a regression problem where data related to each employee represent one **observation**. The presumption is that the experience, education, role, and city are the independent features, while the salary depends on them.

Similarly, you can try to establish a mathematical dependence of the prices of houses on their areas, numbers of bedrooms, distances to the city center, and so on.

Generally, in regression analysis, you usually consider some phenomenon of interest and have a number of observations. Each observation has two or more features. Following the assumption that (at least) one of the features depends on the others, you try to establish a relation among them.

In other words, **you need to find a function that maps some features or variables to others sufficiently well.**

The dependent features are called the **dependent variables, outputs, or responses.**

The independent features are called the **independent variables, inputs, or predictors**.

Regression problems usually have one continuous and unbounded dependent variable. The inputs, however, can be continuous, discrete, or even categorical data such as gender, nationality, brand, and so on.

It is a common practice to denote the outputs with y and inputs with x . If there are two or more independent variables, they can be represented as the vector $\mathbf{x} = (x_1, \dots, x_r)$, where r is the number of inputs.

When Do You Need Regression?

Typically, you need regression to answer whether and how some phenomenon influences the other or **how several variables are related**. For example, you can use it to determine *if* and *to what extent* the experience or gender impact salaries.

Regression is also useful when you want **to forecast a response** using a new set of predictors. For example, you could try to predict electricity consumption of a household for the next hour given the outdoor temperature, time of day, and number of residents in that household.

Regression is used in many different fields: economy, computer science, social sciences, and so on. Its importance rises every day with the availability of large amounts of data and increased awareness of the practical value of data.

Linear Regression

Linear regression is probably one of the most important and widely used regression techniques. It's among the simplest regression methods. One of its main advantages is the ease of interpreting results.

Problem Formulation

When implementing linear regression of some dependent variable y on the set of independent variables $\mathbf{x} = (x_1, \dots, x_r)$, where r is the number of predictors, you assume a linear relationship between y and $\mathbf{x}$: $y = \beta_0 + \beta_1 x_1 + \dots + \beta_r x_r + \varepsilon$. This equation is the **regression equation**. $\beta_0, \beta_1, \dots, \beta_r$ are the **regression coefficients**, and ε is the **random error**.

Linear regression calculates the **estimators** of the regression coefficients or simply the **predicted weights**, denoted with $b_0, b_1, \dots, b_r$. They define the **estimated regression function** $(\mathbf{x}) = b_0 + b_1 x_1 + \dots + b_r x_r$. This function should capture the dependencies between the inputs and output sufficiently well.

The **estimated or predicted response**, $(\mathbf{x}_i)$, for each observation $i = 1, \dots, n$, should be as close as possible to the corresponding **actual response** y_i . The differences $y_i - (\mathbf{x}_i)$ for all observations $i = 1, \dots, n$, are called the **residuals**. Regression is about determining the **best predicted weights**, that is the weights corresponding to the smallest residuals.

To get the best weights, you usually **minimize the sum of squared residuals** (SSR) for all observations $i = 1, \dots, n$: $SSR = \sum_i (y_i - f(\mathbf{x}_i))^2$. This approach is called the **method of ordinary least squares**.

Regression Performance

The variation of actual responses $y_i, i = 1, \dots, n$, occurs partly due to the dependence on the predictors $\mathbf{x}_i$. However, there is also an additional inherent variance of the output.

The **coefficient of determination**, denoted as R^2 , tells you which amount of variation in y can be explained by the dependence on $\mathbf{x}$ using the particular regression model. Larger R^2 indicates a better fit and means that the model can better explain the variation of the output with different inputs.

The value $R^2 = 1$ corresponds to $SSR = 0$, that is to the **perfect fit** since the values of predicted and actual responses fit completely to each other.

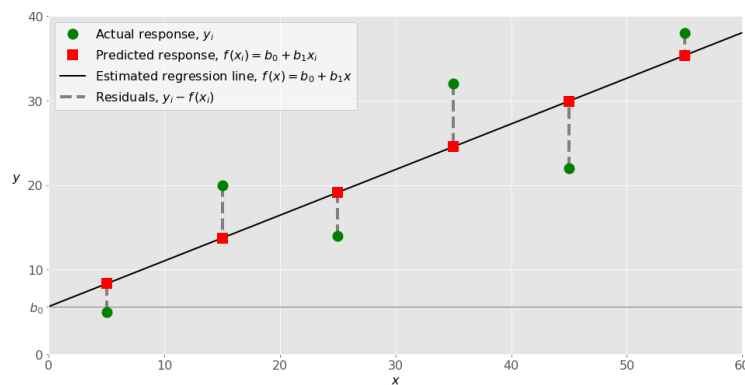
Simple Linear Regression

Simple or single-variate linear regression is the simplest case of linear regression with a single independent variable, $x = x$.

The following figure illustrates simple linear regression:

When implementing simple linear regression, you typically start with a given set of input-output (x - y) pairs (green circles). These pairs are your observations. For example, the leftmost observation (green circle) has the input $x = 5$ and the actual output (response) $y = 5$. The next one has $x = 15$ and $y = 20$, and so on.

The estimated regression function (black line) has the equation $\hat{y}(x) = b_0 + b_1x$. Your goal is to calculate the optimal values of the predicted weights b_0 and b_1 that minimize SSR and determine the estimated regression function. The value of b_0 , also called the **intercept**, shows the point where the estimated regression line crosses the y axis. It is the value of the estimated response $\hat{y}(x)$ for $x = 0$. The value of b_1 determines the **slope** of the estimated regression line.



Example of simple linear regression

The predicted responses (red squares) are the points on the regression line that correspond to the input values. For example, for the input $x = 5$, the predicted response is $\hat{y}(5) = 8.33$ (represented with the leftmost red square).

The residuals (vertical dashed gray lines) can be calculated as $y_i - \hat{y}(x_i) = y_i - b_0 - b_1x_i$ for $i = 1, \dots, n$. They are the distances between the green circles and red squares. When you implement linear regression, you are actually trying to minimize these distances and make the red squares as close to the predefined green circles as possible.

Multiple Linear Regression

Multiple or multivariate linear regression is a case of linear regression with two or more independent variables.

If there are just two independent variables, the estimated regression function is $\hat{y}(x_1, x_2) = b_0 + b_1x_1 + b_2x_2$. It represents a regression plane in a three-dimensional space. The goal of regression is to determine the values of the weights b_0 , b_1 , and b_2 such that this plane is as close as possible to the actual responses and yield the minimal SSR.

The case of more than two independent variables is similar, but more general. The estimated regression function is $\hat{y}(x_1, \dots, x_r) = b_0 + b_1x_1 + \dots + b_rx_r$, and there are $r + 1$ weights to be determined when the number of inputs is r .

Procedure : Implementing Linear Regression in Python

Python Packages for Linear Regression

The package **NumPy** is a fundamental Python scientific package that allows many high-performance operations on single- and multi-dimensional arrays. It also offers many mathematical routines. Of course, it's open source.

The package **scikit-learn** is a widely used Python library for machine learning, built on top of NumPy and some other packages. It provides the means for preprocessing data, reducing dimensionality, implementing regression, classification, clustering, and more. Like NumPy, scikit-learn is also open source.

Simple Linear Regression With scikit-learn

There are five basic steps when you're implementing linear regression:

1. **Import the packages and classes you need.**
 - Numpy
 - Pandas
 - Matplotlib
 - scikit-learn
2. Do the Data Preprocessing using Data Preprocessing Tools
3. Create a regression model and fit it with existing data.
4. Check the results of model fitting to know whether the model is satisfactory.
5. Apply the model for predictions.

These steps are more or less general for most of the regression approaches and implementations.

Task to Perform :

1. Implement Linear Regression on dataset provided and find its accuracy
2. Implement Multiple linear regression and find its Coefficients and R Square metric

Program:

1. Simple Linear Regression

Importing the libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

#Importing the dataset

```
dataset = pd.read_csv('Salary_Data.csv')
dataset.info()
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

#Splitting the dataset into the Training set and Test set

```
from sklearn.model_selection import train_test_split
```

```

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size =
0.2,random_state = 0)
#Training the Simple Linear Regression model on the Training set
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train,y_train)
#Getting the final linear regression equation with the values of the coefficients
print("B1=",regressor.coef_)
print("B0 =",regressor.intercept_)
Predicting the Test set results
y_pred=regressor.predict(X_test)
print(y_test)
print(y_pred)
#Visualising the Training set results
plt.scatter(X_train,y_train,color='red')
plt.plot(X_train,regressor.predict(X_train),color='blue')
# regressor.predict(x_train) because it is predicted salaries for x_train
plt.title("Simple Linear Regression on Training Data")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.show()
#Visualising the Test set results
plt.scatter(X_test,y_test,color='red')
plt.plot(X_test,y_pred,color='blue')
# y_pred because it is predicted salaries for x_test
plt.title("Simple Linear Regression on Testing Data")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.show()
#Evaluating the performance of the model
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)

```

2. Multiple Linear Regression

#Importing the libraries

```

import numpy as np
import matplotlib.pyplot as plt

```

```

import pandas as pd

#Importing the dataset

dataset = pd.read_csv('50_Startups.csv')

#Preparing Data as input and output

X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

#Encoding categorical data

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct= ColumnTransformer ( transformers = [ ( 'encoder' ,
OneHotEncoder(), [3])], remainder='passthrough')
X=np.array (ct.fit_transform(X))

# Splitting the dataset into the Training set and Test set

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size
= 0.2 ,random_state = 0)

# Training the Multiple Linear Regression model on the Training set

from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train,y_train)

# Predicting the Test set results

y_pred = regressor.predict(X_test)

# Making a single prediction (for example the profit of a startup with R&D Spend =
160000, Administration Spend = 130000, Marketing Spend = 300000 and State =
'California')

print(regressor.predict([[1,0,0,160000,130000,300000]]))

#Getting the final linear regression equation with the values of the coefficients

print(regressor.coef_)
print(regressor.intercept_)

# variance score: 1 means perfect prediction

print('Variance score: {}'.format(regressor.score(X_test,
y_test)))

# plot for residual error

plt.style.use('fivethirtyeight')
plt.scatter(regressor.predict(X_train),
regressor.predict(X_train) - y_train,color = "green", s = 10,
label = 'Train data')

```

```
plt.scatter(regressor.predict(X_test), regressor.predict(X_test)
- y_test,color = "blue", s = 10, label = 'Test data')
plt.hlines(y = 0, xmin = 0, xmax = 200000, linewidth = 2)
plt.legend(loc = 'upper right')
plt.title("Residual errors")
plt.xlabel("Profit")
plt.ylabel("Error")
plt.show()
#Find the R^2
from sklearn.metrics import r2_score
print(r2_score(y_test,y_pred))
```

Results:

	Simple Linear Regression	Multiple Linear Regression
Accuracy Score		
R2 Score		
Coefficients		
Intercept		
Test Example Result		

Conclusion: In this way we successfully implement Simple and Multiple linear regression and find its accuracy.

Experiment No. 4

Title : Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample and Visualize model with Python.

Problem Statement : The main aim of this lab is to demonstrate the how to Creating and Visualizing Decision Trees with Python

Secondary aim of this lab is to undersand

- What Decision Tree is
- What Decision Tree is used for
- How to Fit a Decision Tree Model using Scikit-Learn
- How to Visualize Decision Trees using Matplotlib

Software Used: Jupyter Notebook , Python 3.6, Or Google Colab

Dataset Used: bill_authentication.csv

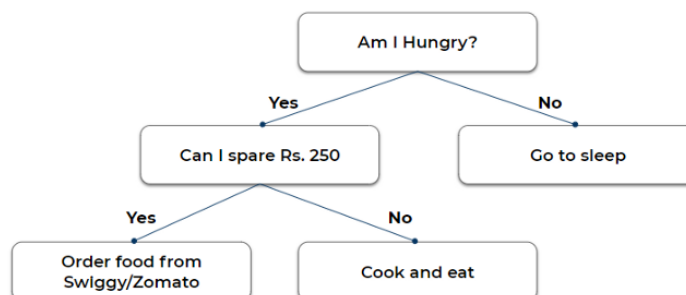
Theory:

Introduction to Decision Tree

Formally a decision tree is a graphical representation of all possible solutions to a decision. These days, tree-based algorithms are the most commonly used algorithms in case of supervised learning scenarios. They are easier to interpret and visualize with great adaptability. We can use tree-based algorithms for both regression and classification problems, However, most of the time they are used for classification problem.

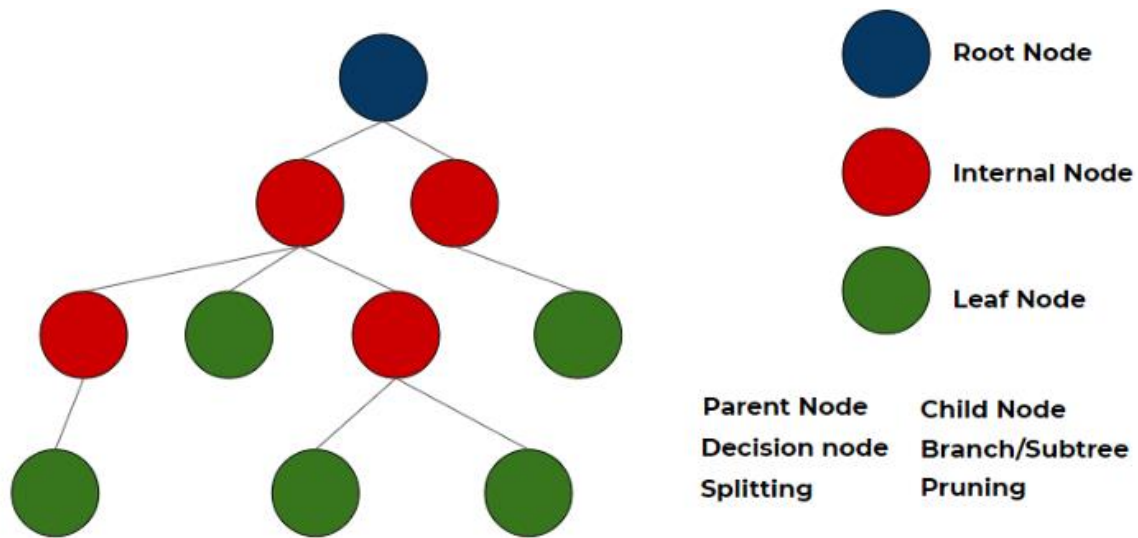
Let's understand a decision tree from an example: Yesterday evening, I skipped dinner at my usual time because I was busy taking care of some stuff. Later in the night, I felt butterflies in my stomach. I thought only if I wasn't hungry, I could have gone to sleep as it is but as that was not the case, I decided to eat something. I had two options, to order something from outside or cook myself.

I figured if I order, I will have to spare at least INR 250 on it. I finally decided to order it anyway as it was pretty late and I was in no mood of cooking. This complete incident can be graphically represented as shown in the following figure.



This representation is nothing but a decision tree.

Terminologies associated with decision tree



- **Parent node:** In any two connected nodes, the one which is higher hierarchically, is a parent node.
- **Child node:** In any two connected nodes, the one which is lower hierarchically, is a child node.
- **Root node:** Starting node from which the tree starts, It has only child nodes. The root node does not have a parent node. (dark blue node in the above image)
- **Leaf Node/leaf:** Nodes at the end of the tree, which do not have any children are leaf nodes or called simply leaf. (green nodes in the above image)
- **Internal nodes/nodes:** All the in-between the root node and the leaf nodes are internal nodes or simply called nodes. internal nodes have both a parent and at least one child. (red nodes in the above image)
- **Splitting:** Dividing a node in two or more sub-nodes or adding two or more children to a node.
- **Decision node:** when a parent splits into two or more children nodes then that node is called a decision node.
- **Pruning:** When we remove the sub-node of a decision node, it is called pruning. You can understand it as the opposite process of splitting.
- **Branch/Sub-tree:** a subsection of the entire tree is called a branch or sub-tree.

Types of Decision Tree

1. Regression Tree

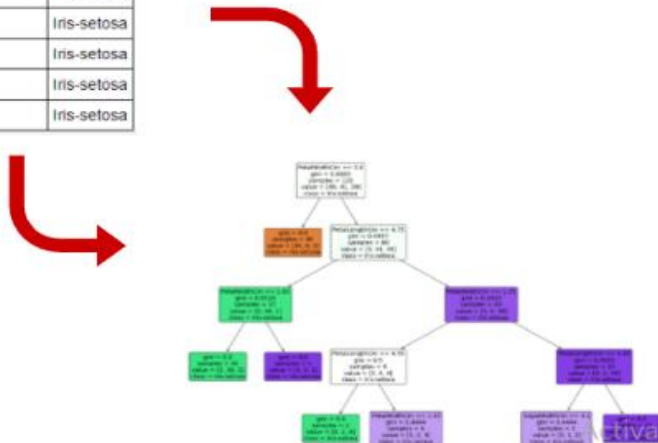
A regression tree is used when the dependent variable is **continuous**. The value obtained by leaf nodes in the training data is the **mean** response of observation falling in that region. Thus, if an unseen data observation falls in that region, its prediction is made with the mean value. This means that even if the dependent variable in training data was continuous, it will only take discrete values in the test set. A regression tree follows a **top-down greedy approach**.

2. Classification Tree

A classification tree is used when the dependent variable is **categorical**. The value obtained by leaf nodes in the training data is the **mode** response of observation falling in that region. It follows a **top-down greedy approach**.

Building a decision Tree from data

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
Id					
1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3.0	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa
5	5.0	3.6	1.4	0.2	Iris-setosa



How to create a tree from tabular data? which feature should be selected as the root node? on what basis should a node be split? To all these questions answer is in this section

The decision of making strategic splits heavily affects a tree's accuracy. The purity of the node should increase with respect to the target variable after each split. The decision tree splits the nodes on all available variables and then selects the split which results in most homogeneous sub-nodes.

The following are the most commonly used algorithms for splitting

1. Gini impurity

Gini says, if we select two items from a population at random then they must be of the same class and the probability for this is 1 if the population is pure.

1. It works with the categorical target variable "Success" or "Failure".
2. It performs only Binary splits
3. Higher the value of Gini higher the homogeneity.
4. CART (Classification and Regression Tree) uses the Gini method to create binary splits.

Steps to Calculate Gini impurity for a split

1. Calculate Gini impurity for sub-nodes, using formula subtracting the sum of the square of probability for success and failure from one.

$$1 - (p^2 + q^2)$$
 where $p = P(\text{Success})$ & $q = P(\text{Failure})$
2. Calculate Gini for split using the weighted Gini score of each node of that split
3. Select the feature with the least Gini impurity for the split.

2. Chi-Square

It is an algorithm to find out the statistical significance between the differences between sub-nodes and parent node. We measure it by the sum of squares of standardized differences between observed and expected frequencies of the target variable.

1. It works with the categorical target variable "Success" or "Failure".

2. It can perform two or more splits.
3. Higher the value of Chi-Square higher the statistical significance of differences between sub-node and Parent node.
4. Chi-Square of each node is calculated using the formula,
5. Chi-square = $((\text{Actual} - \text{Expected})^2 / \text{Expected})^{1/2}$
6. It generates a tree called CHAID (Chi-square Automatic Interaction Detector)

Steps to Calculate Chi-square for a split:

1. Calculate Chi-square for an individual node by calculating the deviation for Success and Failure both
2. Calculated Chi-square of Split using Sum of all Chi-square of success and Failure of each node of the split
3. Select the split where Chi-Square is maximum.

3. Information Gain

A less impure node requires less information to describe it and, a more impure node requires more information. Information theory is a measure to define this degree of disorganization in a system known as Entropy. If the sample is completely homogeneous, then the entropy is zero and if the sample is equally divided (50% — 50%), it has an entropy of one. Entropy is calculated as follows.

$$\text{Entropy} = -p.\log_2^p - q.\log_2^q$$

Steps to calculate entropy for a split:

1. Calculate the entropy of the parent node
2. Calculate entropy of each individual node of split and calculate the weighted average of all sub-nodes available in the split. The lesser the entropy, the better it is.
3. calculate information gain as follows and chose the node with the highest information gain for splitting

$$\text{Information Gain} = 1 - \text{Entropy}$$

4. Reduction in Variance

Till now, we have discussed the algorithms for the categorical target variable. Reduction in variance is an algorithm used for continuous target variables (regression problems).

1. Used for continuous variables
2. This algorithm uses the standard formula of variance to choose the best split.
3. The split with lower variance is selected as the criteria to split the population

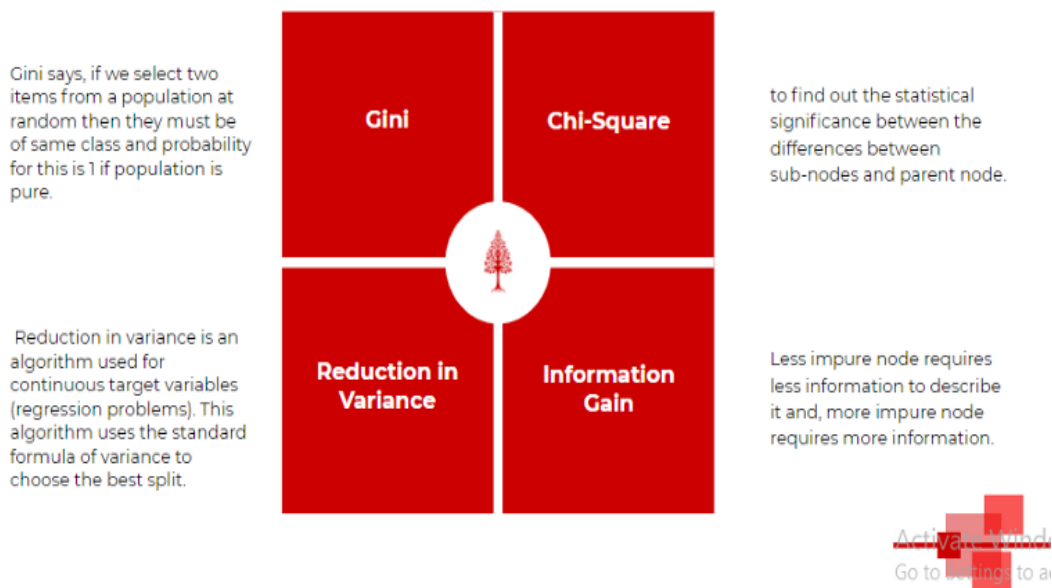
Steps to calculate Variance:

1. Calculate variance for each node.

$$\sigma^2 = \frac{\sum_{i=1}^N (x_i - \mu)^2}{N}$$

1. Calculate variance for each split as a weighted average of each node variance.
2. The node with lower variance is selected as the criteria to split.

In summary:



Implementing Decision Tree in Python

Step-1: Importing the packages

Our primary packages involved in building our model are pandas, scikit-learn, and NumPy.

Step-2: Importing data and EDA

In this step, we will be utilizing the 'Pandas' package available in python to import and do some EDA on it.

The dataset we will be using to build our decision tree model is a **bill_authentication**

Now we have a clear idea of our dataset. After importing the data, let's get some basic information on the data using the 'info' function. The information provided by this function includes the number of entries, index number, column names, non-null values count, attribute type, etc.

Step-3: Data Processing

Step-4: Splitting the data

After processing our data to be of the right structure, we are now set to define the 'X' variable or the independent variable and the 'Y' variable or the dependent variable.

Step-5: Building the model & Predictions

Building a decision tree can be feasibly done with the help of the ‘DecisionTreeClassifier’ algorithm provided by the scikit-learn package. After that, we can make predictions of our data using our trained model. Finally, the precision of our predicted results can be calculated using the ‘accuracy_score’ evaluation metric.

Step-6: Visualizing the model

Now that we have our decision tree model and let’s visualize it by utilizing the ‘plot_tree’ function provided by the scikit-learn package in python. Follow the code to produce a beautiful tree diagram out of your decision tree model in python.

Task to Perform:

1. Implement Decision tree and find the accuracy of your model
2. Visualize your build model in python

Code:**Importing the libraries**

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Importing the dataset

```
dataset = pd.read_csv("bill_authentication.csv")
```

Preparing Data into Input Feature and Output Target Class

```
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

Splitting the dataset into the Training set and Test set

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split (X, y, test_size =
0.20 , random_state=0)
```

Feature Scaling

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test =sc.fit_transform(X_test)
```

Training the Decision Tree Classification model on the Training set

```
from sklearn.tree import DecisionTreeClassifier
classifier = DecisionTreeClassifier(criterion = 'entropy',
random_state = 0)
classifier.fit(X_train, y_train)
```

Predicting the Test set results

```
y_pred = classifier.predict(X_test)
```

```

print(np.concatenate((y_pred.reshape(len(y_pred),1),y_test.reshape(
len(y_test),1)),1))

# Making the Confusion Matrix

from sklearn.metrics import confusion_matrix,accuracy_score
print(y_test.shape)
cm = confusion_matrix(y_test, y_pred)
print(cm)
accuracy_score (y_test, y_pred)

from sklearn.metrics import classification_report
print(classification_report(y_test,y_pred))

# Visualising the Training set results

# VISUALIZE TEXT REPRESENTATION

from sklearn import tree
text_representation = tree.export_text(classifier)
print(text_representation)

# Visualizing Tree

fig = plt.figure(figsize=(25,20))
tree.plot_tree(classifier)

```

Result:

Accuracy	
Precision	
Recall	
F1 Score	

Conclusion: In this way we successfully implement and visualize Decision Tree in python

Experiment No. 5

Title : Write a program to implement k-Nearest Neighbor algorithm to classify the iris data set. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.

Problem Statement: The main aim of this lab is to demonstrate the how to Creating and Visualizing KNN with Python

Secondary aim of this lab is to understand

- What is KNN?
- What KNN is used for
- How to Fit a KNN using Scikit-Learn
- How to Visualize KNN using Matplotlib

Software Used: Jupyter Notebook , Python 3.6, Or Google Colab

Dataset Used: iris.data (Online)

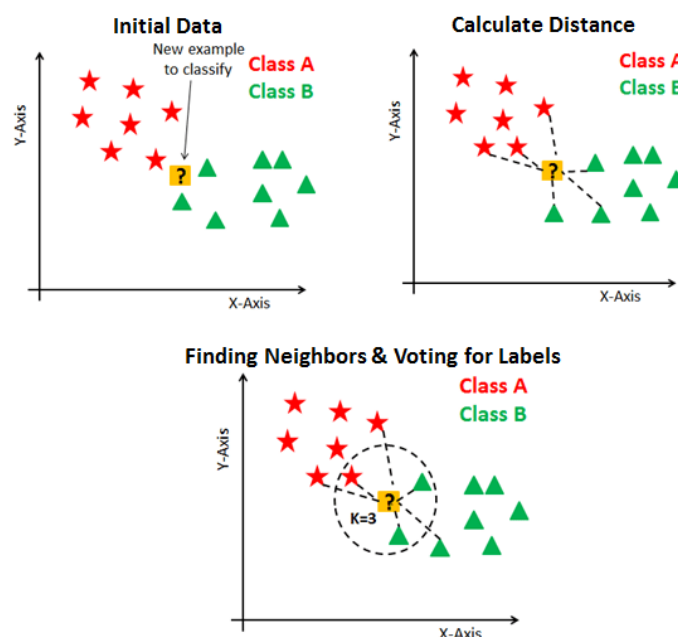
Theory:

Introduction to K Nearest Neighbor

K Nearest Neighbor(KNN) algorithm is a very simple, easy to understand, versatile and one of the topmost machine learning algorithms. In k-NN classification, the output is a class membership. An object is classified by a plurality vote of its neighbours, with the object being assigned to the class most common among its k nearest neighbours (k is a positive integer, typically small). If $k = 1$, then the object is simply assigned to the class of that single nearest neighbour.

In KNN, K is the number of nearest neighbours. The number of neighbours is the core deciding factor. K is generally an odd number if the number of classes is 2. When $K=1$, then the algorithm is known as the nearest neighbour algorithm. This is the simplest case.

Suppose P1 is the point, for which label needs to be predicted.



Algorithm steps

STEP 1: Choose the number K of neighbors

STEP 2: Take the K nearest neighbors of the new data point, according to your distance metric

STEP 3: Among these K neighbors, count the number of data points to each category

STEP 4: Assign the new data point to the category where you counted the most neighbors

Pros and Cons of KNN**Pros**

1. It is extremely easy to implement
2. As said earlier, it is lazy learning algorithm and therefore requires no training prior to making real time predictions. This makes the KNN algorithm much faster than other algorithms that require training e.g SVM, linear regression, etc.
3. Since the algorithm requires no training before making predictions, new data can be added seamlessly.
4. There are only two parameters required to implement KNN i.e. the value of K and the distance function (e.g. Euclidean or Manhattan etc.)

Cons

1. The KNN algorithm doesn't work well with high dimensional data because with large number of dimensions, it becomes difficult for the algorithm to calculate distance in each dimension.
2. The KNN algorithm has a high prediction cost for large datasets. This is because in large datasets the cost of calculating distance between new point and each existing point becomes higher.
3. Finally, the KNN algorithm doesn't work well with categorical features since it is difficult to find the distance between dimensions with categorical features.

Procedure:**Step 1: Importing Libraries****Step 2: Importing the Dataset**

We are going to use the famous iris data set for our KNN example. The dataset consists of four attributes: sepal-width, sepal-length, petal-width and petal-length. These are the attributes of specific types of iris plant. The task is to predict the class to which these plants belong. There are three classes in the dataset: Iris-setosa, Iris-versicolor and Iris-virginica.

Step 3: Perform Exploratory Data Analysis**Step 4: Preprocessing****A. Train Test Split**

To avoid over-fitting, we will divide our dataset into training and test splits, which gives us a better idea as to how our algorithm performed during the testing phase. This way our algorithm is tested on un-seen data, as it would be in a production application.

B. Feature Scaling

Before making any actual predictions, it is always a good practice to scale the features so that all of them can be uniformly evaluated.

Step5: Training and Predictions

It is extremely straight forward to train the KNN algorithm and make predictions with it, especially when using Scikit-Learn.

The first step is to import the KNeighborsClassifier class from the sklearn.neighbors library. In the second line, this class is initialized with one parameter, i.e. n_neighbours. This is basically the value for the K. There is no ideal value for K and it is selected after testing and evaluation, however to start out, 5 seems to be the most commonly used value for KNN algorithm.

The final step is to make predictions on our test data.

Step 6: Evaluating the Algorithm

For evaluating an algorithm, confusion matrix, precision, recall and f1 score are the most commonly used metrics. The confusion_matrix and classification_report methods of the sklearn.metrics can be used to calculate these metrics.

The results show that our KNN algorithm was able to classify all the 30 records in the test set with 100% accuracy, which is excellent. Although the algorithm performed very well with this dataset, don't expect the same results with all applications. As noted earlier, KNN doesn't always perform as well with high-dimensionality or categorical features.

Step 7: Comparing Error Rate with the K Value

In the training and prediction section we said that there is no way to know beforehand which value of K that yields the best results in the first go. We randomly chose 5 as the K value and it just happen to result in 100% accuracy.

One way to help you find the best value of K is to plot the graph of K value and the corresponding error rate for the dataset.

In this section, we will plot the mean error for the predicted values of test set for all the K values between 1 and 40.

To do so, let's first calculate the mean of error for all the predicted values where K ranges from 1 and 40.

Step 8: The next step is to plot the error values against K values. Execute the following script to create the plot

Step 9: Visualize the KNN

Task to Perform:

1. Implement Decision tree and find the accuracy of your model
2. Visualize your build model in python

Code:

Importing the libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Importing the dataset

```
url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/iris/iris.data"
```

Assign column names to the dataset

```
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-
width', 'Class']
```

Read dataset to pandas dataframe

```
dataset = pd.read_csv(url, names=names)
```

Seprating Dependent and Independent Variables

```
X = dataset.iloc[:, :-1].values
```

```
y = dataset.iloc[:, -1].values
```

Splitting the dataset into the Training set and Test set

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20, random_state = 5)
```

Feature Scaling

```
from sklearn.preprocessing import StandardScaler
```

```
sc = StandardScaler()
```

```
sc.fit(X_train)
```

```
X_train = sc.transform(X_train)
```

```
X_test = sc.transform(X_test)
```

Training the K-NN model on the Training set

```
from sklearn.neighbors import KNeighborsClassifier
```

```
classifier = KNeighborsClassifier (n_neighbors=10 ,p=2 ,  
metric='minkowski')
```

```
classifier.fit(X_train,y_train)
```

Predicting the Test set results

```
y_pred = classifier.predict(X_test)
```

```
print(np.concatenate((y_pred.reshape(len(y_pred),1),y_test.reshape(  
len(y_test),1)),1))
```

Evaluating the Algorithm**# Making the Confusion Matrix & Predicting Accuracy Score**

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
cm = confusion_matrix(y_test,y_pred)
```

```
print(cm)
```

```
accuracy = accuracy_score(y_test, y_pred)*100
```

```
print('Accuracy of our model is equal ' + str(round(accuracy, 2)) +  
' %.')
```

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, y_pred))
```

#Comparing Error Rate with the K Value**#Parameter Tuning Using**

#cross-validation for parameter tuning:

```
from sklearn.model_selection import cross_val_score
```

creating list of K for KNN

```
k_list = list(range(1,50))
```

creating list of cv scores

```
cv_scores = []
```

perform 10-fold cross validation

```
for k in k_list:
```

```
    knn = KNeighborsClassifier(n_neighbors=k)
```

```
    scores = cross_val_score(knn, X_train, y_train, cv=10,
scoring='accuracy')
```

```
    cv_scores.append(scores.mean())
```

plot the error values against K values

```
import seaborn as sns
```

changing to misclassification error

```
MSE = [1-x for x in cv_scores]
```

```
plt.figure()
```

```
plt.figure(figsize=(15,10))
```

```
plt.title('The optimal number of neighbors', fontsize=20,
fontweight='bold')
```

```
plt.xlabel('Number of Neighbors K', fontsize=15)
```

```
plt.ylabel('Misclassification Error', fontsize=15)
```

```
sns.set_style("whitegrid")
```

```
plt.plot(k_list, MSE)
```

```
plt.show()
```

finding best k

```
best_k = k_list[MSE.index(min(MSE))]
```

```
print("The optimal number of neighbors is %d." % best_k)
```

Visualize Test Result of KNN

```
from matplotlib.colors import ListedColormap
```

```
markers = ('s', 'x', 'o')
```

```
colors = ('red', 'blue', 'black')
```

```
cmap = ListedColormap(colors[:len(np.unique(y_test))])
```

```
for idx, cl in enumerate(np.unique(y)):
```

```
    plt.scatter(x=X[y == cl, 0], y=X[y == cl, 1], c=cmap(idx),
marker=markers[idx], label=cl)
```

Results:

Best Value of K	
Training Accuracy	
Precision	
Class for Testing Example	
F1 Score	

Conclusion: In this way we successfully implement and visualize KNN in python

Experiment No. 6

Title : Write a program to implement the Kernel SVM Algorithm for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.

Problem Statement: The main aim of this lab is to demonstrate the how to Creating and Visualizing SVM algorithm with Python

Secondary aim of this lab is to understand

- What is SVM?
- What SVM is used for
- How to Fit a SVM using Scikit-Learn
- How to Visualize SVM using Matplotlib

Software Used: Jupyter Notebook , Python 3.6, Or Google Colab

Dataset Used: iris

Theory:

Generally, Support Vector Machines is considered to be a classification approach, it but can be employed in both types of classification and regression problems. It can easily handle multiple continuous and categorical variables. SVM constructs a hyperplane in multidimensional space to separate different classes. SVM generates optimal hyperplane in an iterative manner, which is used to minimize an error. The core idea of SVM is to find a maximum marginal hyperplane(MMH) that best divides the dataset into classes.

Support Vectors

Support vectors are the data points, which are closest to the hyperplane. These points will define the separating line better by calculating margins. These points are more relevant to the construction of the classifier.

Hyperplane

A hyperplane is a decision plane which separates between a set of objects having different class memberships.

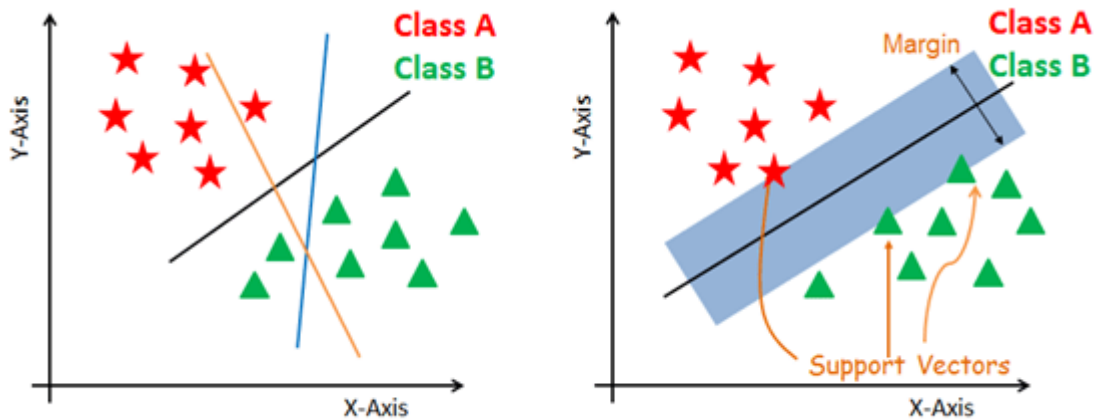
Margin

A margin is a gap between the two lines on the closest class points. This is calculated as the perpendicular distance from the line to support vectors or closest points. If the margin is larger in between the classes, then it is considered a good margin, a smaller margin is a bad margin.

How does SVM work?

The main objective is to segregate the given dataset in the best possible way. The distance between the either nearest points is known as the margin. The objective is to select a hyperplane with the maximum possible margin between support vectors in the given dataset. SVM searches for the maximum marginal hyperplane in the following steps:

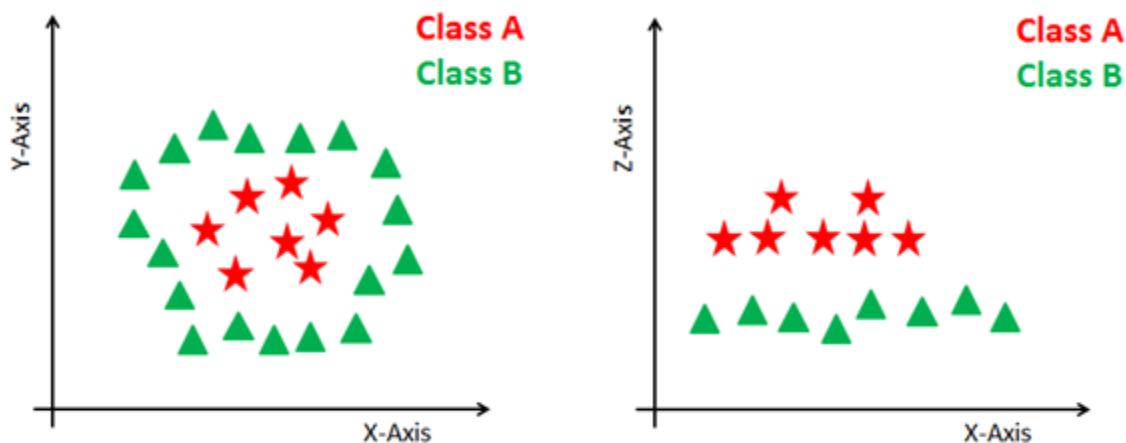
1. Generate hyperplanes which segregates the classes in the best way. Left-hand side figure showing three hyperplanes black, blue and orange. Here, the blue and orange have higher classification error, but the black is separating the two classes correctly.
2. Select the right hyperplane with the maximum segregation from the either nearest data points as shown in the right-hand side figure.



Dealing with non-linear and inseparable planes

Some problems can't be solved using linear hyperplane, as shown in the figure below (left-hand side).

In such situation, SVM uses a kernel trick to transform the input space to a higher dimensional space as shown on the right. The data points are plotted on the x-axis and z-axis (Z is the squared sum of both x and y: $z=x^2+y^2$). Now you can easily segregate these points using linear separation.



SVM Kernels

The SVM algorithm is implemented in practice using a kernel. A kernel transforms an input data space into the required form. SVM uses a technique called the kernel trick. Here, the kernel takes a low-dimensional input space and transforms it into a higher dimensional space. In other words, you can say that it converts nonseparable problem to separable problems by adding more dimension to it. It is most useful in non-linear separation problem. Kernel trick helps you to build a more accurate classifier.

Linear Kernel

A linear kernel can be used as normal dot product any two given observations. The product between two vectors is the sum of the multiplication of each pair of input values.

$$K(x, x_i) = \text{sum}(x * x_i)$$

Polynomial Kernel

A polynomial kernel is a more generalized form of the linear kernel. The polynomial kernel can distinguish curved or nonlinear input space.

$$K(x, x_i) = 1 + \sum (x * x_i)^d$$

Where d is the degree of the polynomial. d=1 is similar to the linear transformation. The degree needs to be manually specified in the learning algorithm.

Radial Basis Function Kernel

The Radial basis function kernel is a popular kernel function commonly used in support vector machine classification. RBF can map an input space in infinite dimensional space.

$$K(x, x_i) = \exp(-\gamma * \sum ((x - x_i)^2))$$

Here gamma is a parameter, which ranges from 0 to 1.

A higher value of gamma will perfectly fit the training dataset, which causes over-fitting. Gamma=0.1 is considered to be a good default value. The value of gamma needs to be manually specified in the learning algorithm.

Procedure:**Step 1: Importing Libraries****Step 2: Importing the Dataset**

We are going to use the famous iris data set for our SVM example. The dataset consists of four attributes: sepal-width, sepal-length, petal-width and petal-length. These are the attributes of specific types of iris plant. The task is to predict the class to which these plants belong. There are three classes in the dataset: Iris-setosa, Iris-versicolor and Iris-virginica. Out of four attributes we are using only two for better visualization i.e petal-width and petal-length.

Step 3: Perform Exploratory Data Analysis**Step 4: Preprocessing****C. Train Test Split**

To avoid over-fitting, we will divide our dataset into training and test splits, which gives us a better idea as to how our algorithm performed during the testing phase. This way our algorithm is tested on un-seen data, as it would be in a production application. Here we keep 30% data for testing and 70% data for training.

D. Feature Scaling

Before making any actual predictions, it is always a good practice to scale the features so that all of them can be uniformly evaluated.

Step 5: Plot the original Data**Step 6: Training and Predictions**

It is extremely straight forward to train the SVM algorithm and make predictions with it, especially when using Scikit-Learn.

The first step is to import the SVC class from the sklearn.svm library. In the second line, this class is initialized with one parameter, i.e. kernel='rbf', random_state=0, gamma=.10, C=1.0.

Step 7: Finding Accuracy of model on Test and Train Set**Step 8: Create the function for Visualizing Testing and Training model****Step 9: Visualize the Train set results****Step 10: Visualize the Test set results**

Task To perform:

1. Implement SVM and find the accuracy of your model
2. Visualize your build model in python

Code:**# Importing the libraries**

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn import datasets
%pylab inline
pylab.rcParams['figure.figsize'] = (10, 6)
```

importing the dataset

```
iris = datasets.load_iris()
print(iris)

# We'll use the petal length and width only for this analysis
X = iris.data[:, [2, 3]]
y = iris.target
```

#Exploratory Data Analysis**#Place the iris data into a pandas dataframe**

```
iris_df = pd.DataFrame(iris.data[:, [2, 3]],
columns=iris.feature_names[2:])
```

View the first 5 rows of the data

```
print(iris_df.head())
```

Print the unique labels of the dataset

```
print('\n' + 'The unique labels in this data are ' +
str(np.unique(y)))
```

Splitting the dataset into the Training set and Test set

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=.3, random_state=0)
print('There are {} samples in the training set and {} samples in
the test set'.format(
X_train.shape[0], X_test.shape[0]))
```

Feature Scaling

```
from sklearn.preprocessing import StandardScaler
```



```

sc = StandardScaler()
sc.fit(X_train)
X_train_std = sc.transform(X_train)
X_test_std = sc.transform(X_test)
print('After standardizing our features, the first 5 rows of our
data now look like this:\n')
print(pd.DataFrame(X_train_std, columns=iris_df.columns).head())
# Plot the original Data
from matplotlib.colors import ListedColormap
import matplotlib.pyplot as plt
markers = ('s', 'x', 'o')
colors = ('red', 'blue', 'lightgreen')
cmap = ListedColormap(colors[:len(np.unique(y_test))])
for idx, cl in enumerate(np.unique(y)):
    plt.scatter(x=X[y == cl, 0], y=X[y == cl, 1],
                c=cmap(idx), marker=markers[idx], label=cl)
# Training the SVM model on the Training set
from sklearn.svm import SVC
svm = SVC(kernel='rbf', random_state=0, gamma=.10, C=1.0)
svm.fit(X_train_std, y_train)
# Finding Accuracy of model on Test and Train Set
print('The accuracy of the svm classifier on training data is
{:.2f} out of 1'.format(svm.score(X_train_std, y_train)))
print('The accuracy of the svm classifier on test data is {:.2f}
out of 1'.format(svm.score(X_test_std, y_test)))
# Create the function for Visualizing Testing and Training model
def versiontuple(v):
    return tuple(map(int, (v.split("."))))
def plot_decision_regions(X, y, classifier, test_idx=None,
resolution=0.02):
    # setup marker generator and color map
    markers = ('s', 'x', 'o', '^', 'v')
    colors = ('red', 'blue', 'lightgreen', 'gray', 'cyan')
    cmap = ListedColormap(colors[:len(np.unique(y))])

    # plot the decision surface
    x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1

```

```

xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
                        np.arange(x2_min, x2_max, resolution))
Z = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
Z = Z.reshape(xx1.shape)
plt.contourf(xx1, xx2, Z, alpha=0.4, cmap=cmap)
plt.xlim(xx1.min(), xx1.max())
plt.ylim(xx2.min(), xx2.max())

for idx, cl in enumerate(np.unique(y)):
    plt.scatter(x=X[y == cl, 0], y=X[y == cl, 1],
                alpha=0.8, c=cmap(idx),
                marker=markers[idx], label=cl)

plt.show()

# Visualising the Train set results
plot_decision_regions(X_train_std, y_train, svm)
# Visualising the Test set results
plot_decision_regions(X_test_std, y_test, svm)

```

Results:

Training Accuracy of Model	
Testing Accuracy of Model	
Accuracy of Model if we use Kernel = Polynomial	
Class of Testing Example	

Conclusion: In this way we successfully implement and visualize SVM in python and find its accuracy

Experiment No. 7

Title: Write a program to implement the Random Forest Algorithm for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.

Problem Statement: The main aim of this lab is to demonstrate the how to Creating and Visualizing Random Forest algorithm with Python

Secondary aim of this lab is to understand

- What is Ensemble Learning?
- What is Random Forest and its used for
- How to Fit a Random Forest using Scikit-Learn
- How to Visualize Random Forest using Matplotlib

Software Used: Jupyter Notebook , Python 3.6, Or Google Colab

Dataset Used: heart.csv

Theory:

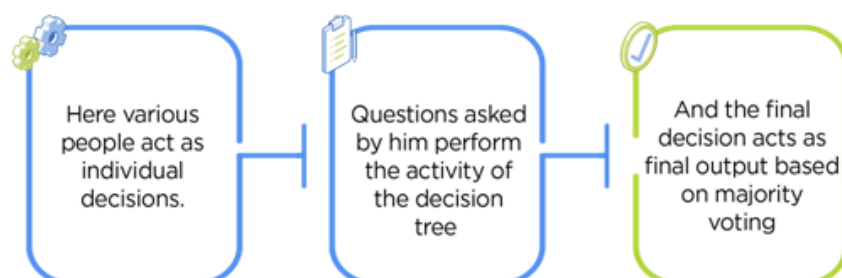
What is Random Forest Algorithm?

Random Forest is one of the most popular and commonly used algorithms by Data Scientists. Random forest is a Supervised Machine Learning Algorithm that is used widely in Classification and Regression problems. It builds decision trees on different samples and takes their majority vote for classification and average in case of regression.

One of the most important features of the Random Forest Algorithm is that it can handle the data set containing continuous variables, as in the case of regression, and categorical variables, as in the case of classification. It performs better for classification and regression tasks. In this tutorial, we will understand the working of random forest and implement random forest on a classification task.

Real-Life Analogy of Random Forest

Let's dive into a real-life analogy to understand this concept further. A student named X wants to choose a course after his 10+2, and he is confused about the choice of course based on his skill set. So he decides to consult various people like his cousins, teachers, parents, degree students, and working people. He asks them varied questions like why he should choose, job opportunities with that course, course fee, etc. Finally, after consulting various people about the course he decides to take the course suggested by most people.

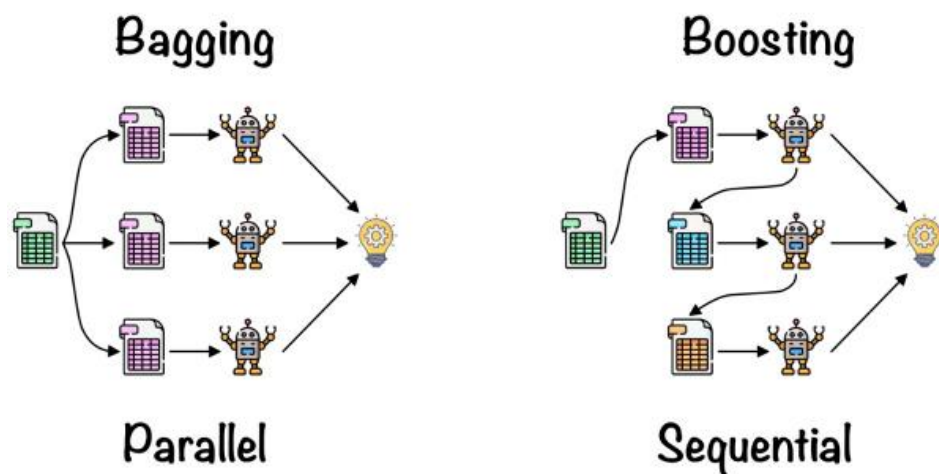


Working of Random Forest Algorithm

Before understanding the working of the random forest algorithm in machine learning, we must look into the ensemble learning technique. Ensemble simply means combining multiple models. Thus a collection of models is used to make predictions rather than an individual model.

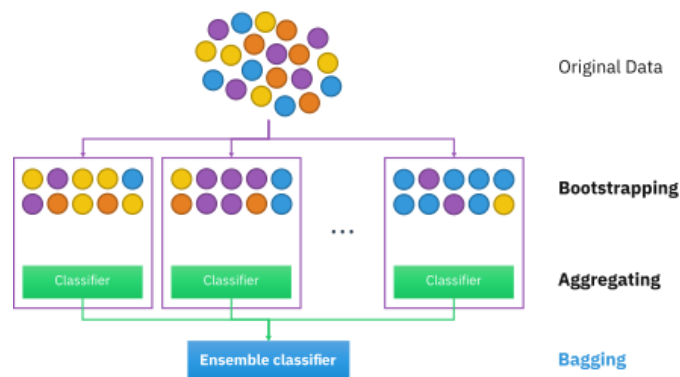
Ensemble uses two types of methods:

- 1. Bagging**– It creates a different training subset from sample training data with replacement & the final output is based on majority voting. For example, Random Forest.
- 2. Boosting**– It combines weak learners into strong learners by creating sequential models such that the final model has the highest accuracy. For example, ADA BOOST, XG BOOST.



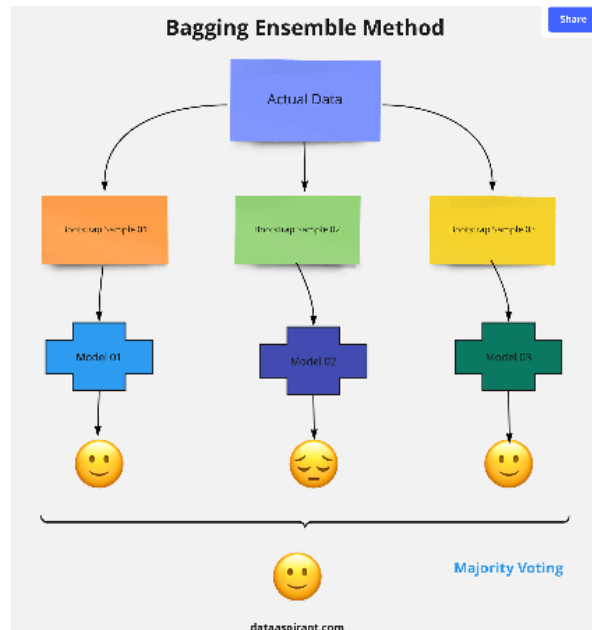
Bagging

Bagging, also known as Bootstrap Aggregation, is the ensemble technique used by random forest. Bagging chooses a random sample/random subset from the entire data set. Hence each model is generated from the samples (Bootstrap Samples) provided by the Original Data with replacement known as row sampling. This step of row sampling with replacement is called bootstrap. Now each model is trained independently, which generates results. The final output is based on majority voting after combining the results of all models. This step which involves combining all the results and generating output based on majority voting, is known as aggregation.



Now let's look at an example by breaking it down with the help of the following figure. Here the bootstrap sample is taken from actual data (Bootstrap sample 01, Bootstrap sample 02, and Bootstrap sample 03) with a replacement which means there is a high possibility that each sample won't contain unique data. The model (Model 01, Model 02, and Model 03) obtained from this bootstrap sample is trained independently.

Each model generates results as shown. Now the Happy emoji has a majority when compared to the Sad emoji. Thus based on majority voting final output is obtained as Happy emoji.



Boosting

Boosting is one of the techniques that use the concept of ensemble learning. A boosting algorithm combines multiple simple models (also known as weak learners or base estimators) to generate the final output. It is done by building a model by using weak models in series.

There are several boosting algorithms; AdaBoost was the first really successful boosting algorithm that was developed for the purpose of binary classification. AdaBoost is an abbreviation for Adaptive Boosting and is a prevalent boosting technique that combines multiple “weak classifiers” into a single “strong classifier.” There are Other Boosting techniques. For more, you can visit

Steps Involved in Random Forest Algorithm

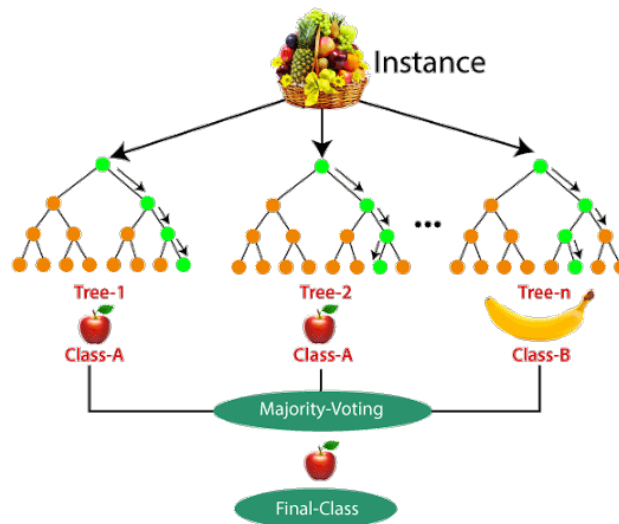
Step 1: In the Random forest model, a subset of data points and a subset of features is selected for constructing each decision tree. Simply put, n random records and m features are taken from the data set having k number of records.

Step 2: Individual decision trees are constructed for each sample.

Step 3: Each decision tree will generate an output.

Step 4: Final output is considered based on Majority Voting or Averaging for Classification and regression, respectively.

For example: consider the fruit basket as the data as shown in the figure below. Now n number of samples are taken from the fruit basket, and an individual decision tree is constructed for each sample. Each decision tree will generate an output, as shown in the figure. The final output is considered based on majority voting. In the below figure, you can see that the majority decision tree gives output as an apple when compared to a banana, so the final output is taken as an apple.



Important Features of Random Forest

Diversity: Not all attributes/variables/features are considered while making an individual tree; each tree is different.

Immune to the curse of dimensionality: Since each tree does not consider all the features, the feature space is reduced.

Parallelization: Each tree is created independently out of different data and attributes. This means we can fully use the CPU to build random forests.

Train-Test split: In a random forest, we don't have to segregate the data for train and test as there will always be 30% of the data which is not seen by the decision tree.

Stability: Stability arises because the result is based on majority voting/ averaging.

Difference Between Decision Tree and Random Forest

Random forest is a collection of decision trees; still, there are a lot of differences in their behavior.

Decision trees	Random Forest
1. Decision trees normally suffer from the problem of overfitting if it's allowed to grow without any control.	1. Random forests are created from subsets of data, and the final output is based on average or majority ranking; hence the problem of overfitting is taken care of.
2. A single decision tree is faster in computation.	2. It is comparatively slower.
3. When a data set with features is taken as input by a decision tree, it will formulate some rules to make predictions.	3. Random forest randomly selects observations, builds a decision tree, and takes the average result. It doesn't use any set of formulas.

Thus random forests are much more successful than decision trees only if the trees are diverse and acceptable.

Important Hyperparameters in Random Forest

Hyperparameters are used in random forests to either enhance the performance and predictive power of models or to make the model faster.

Hyperparameters to Increase the Predictive Power

n_estimators: Number of trees the algorithm builds before averaging the predictions.

max_features: Maximum number of features random forest considers splitting a node.

mini_sample_leaf: Determines the minimum number of leaves required to split an internal node.

criterion: How to split the node in each tree? (Entropy/Gini impurity/Log Loss)

max_leaf_nodes: Maximum leaf nodes in each tree

Hyperparameters to Increase the Speed

n_jobs: it tells the engine how many processors it is allowed to use. If the value is 1, it can use only one processor, but if the value is -1, there is no limit.

random_state: controls randomness of the sample. The model will always produce the same results if it has a definite value of random state and has been given the same hyperparameters and training data.

oob_score: OOB means out of the bag. It is a random forest cross-validation method. In this, one-third of the sample is not used to train the data; instead used to evaluate its performance. These samples are called out-of-bag samples.

Advantages and Disadvantages of Random Forest Algorithm

Advantages

1. It can be used in classification and regression problems.
2. It solves the problem of overfitting as output is based on majority voting or averaging.
3. It performs well even if the data contains null/missing values.
4. Each decision tree created is independent of the other; thus, it shows the property of parallelization.
5. It is highly stable as the average answers given by a large number of trees are taken.
6. It maintains diversity as all the attributes are not considered while making each decision tree though it is not true in all cases.
7. It is immune to the curse of dimensionality. Since each tree does not consider all the attributes, feature space is reduced.
8. We don't have to segregate data into train and test as there will always be 30% of the data, which is not seen by the decision tree made out of bootstrap.

Disadvantages

1. Random forest is highly complex compared to decision trees, where decisions can be made by following the path of the tree.
2. Training time is more than other models due to its complexity. Whenever it has to make a prediction, each decision tree has to generate output for the given input data.

Task To perform:

1. Implement Random Forest and find the accuracy of your model
2. Visualize your build model in python

Code :**# Importing the required libraries**

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

df = pd.read_csv('heart.csv')
# Putting Feature Variable to X and Target variable to y.
# Putting feature variable to X
X = df.drop('heart disease',axis=1)
# Putting response variable to y
y = df['heart disease']
# Train-Test-Split is performed
# now lets split the data into train and test
from sklearn.model_selection import train_test_split
# Splitting the data into train and test
X_train, X_test, y_train, y_test = train_test_split(X, y,
train_size=0.7, random_state=42)
X_train.shape, X_test.shape
# Let's import RandomForestClassifier and fit the data.
from sklearn.ensemble import RandomForestClassifier
classifier_rf = RandomForestClassifier(random_state=42, n_jobs=-1,
max_depth=5,n_estimators=100, oob_score=True)
classifier_rf.fit(X_train, y_train)
# checking the oob score
classifier_rf.oob_score_
# Let's do hyperparameter tuning for Random Forest using
GridSearchCV and fit the data.
rf = RandomForestClassifier(random_state=42, n_jobs=-1)
params = {
    'max_depth': [2,3,5,10,20],
    'min_samples_leaf': [5,10,20,50,100,200],

```



```

    'n_estimators': [10,25,30,50,100,200]
}
from sklearn.model_selection import GridSearchCV
# Instantiate the grid search model
grid_search = GridSearchCV(estimator=rf,param_grid=params,cv = 4,
                           n_jobs=-1, verbose=1,scoring="accuracy")
grid_search.fit(X_train, y_train)
grid_search.best_score_
rf_best = grid_search.best_estimator_
rf_best
# Now, let's visualize
from sklearn.tree import plot_tree
plt.figure(figsize=(80,40))
plot_tree(rf_best.estimators_[5], feature_names =
X.columns,class_names=['Disease', "No Disease"],filled=True);
from sklearn.tree import plot_tree
plt.figure(figsize=(80,40))
plot_tree(rf_best.estimators_[7], feature_names =
X.columns,class_names=['Disease', "No Disease"],filled=True);
# Now let's sort the data with the help of feature importance
rf_best.feature_importances_
## feature importance
imp_df = pd.DataFrame({
    "Varname": X_train.columns,
    "Imp": rf_best.feature_importances_
})
imp_df.sort_values(by="Imp", ascending=False)

```

Results:

oob_score of Classifier	
Accuracy score of Best Model	
Features of Best Estimator	
Feature Importance	Age = Sex =

	Cholestrol =
	BP =

Conclusion: In this way we successfully implement and visualize Random Forest in python and find its accuracy

Experiment No. 8

Title: Write a program to implement the K-means Clustering Algorithm for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.

Problem Statement :

The main aim of this lab is to demonstrate the how to K-means Clustering with Python

Secondary aim of this lab is to understand

- What is K-means Clustering Algorithm?
- How K-means Clustering works?
- How to Choosing the right number of clusters?
- How to Fit a K-means Clustering using Scikit-Learn?
- How to Visualize K-means clustering using Matplotlib?

Software Used : Jupyter Notebook , Python 3.6, Or Google Colab

Theory:

Introduction to K-means Clustering

K-Means Clustering is an unsupervised machine learning algorithm. In contrast to traditional supervised machine learning algorithms, K-Means attempts to classify data without having first been trained with labeled data. Once the algorithm has been run and the groups are defined, any new data can be easily assigned to the most relevant group.

The real world applications of K-Means include:

1. Customer profiling
2. Market segmentation
3. Computer vision
4. Search engines
5. Astronomy

How it works

1. Select K (i.e. 2) random points as cluster centers called centroids
2. Assign each data point to the closest cluster by calculating its distance with respect to each centroid
3. Determine the new cluster center by computing the average of the assigned points
4. Repeat steps 2 and 3 until none of the cluster assignments change

Choosing the right number of clusters

Often times the data you'll be working with will have multiple dimensions making it difficult to visual. As a consequence, the optimum number of clusters is no longer obvious. Fortunately, we have a way of determining this mathematically.

We graph the relationship between the number of clusters and Within Cluster Sum of Squares (WCSS) then we select the number of clusters where the change in WCSS begins to level off (elbow method).

WCSS is defined as the sum of the squared distance between each member of the cluster and its centroid.

For example, the computed WCSS for figure 1 would be greater than the WCSS calculated for figure 1.

Procedure:

1. Import the required libraries.
2. Import the Dataset.
3. Using the elbow method to find the optimal number of clusters
Even though we already know the optimal number of clusters, I figured we could still benefit from determining it using the elbow method. To get the values used in the graph, we train multiple models using a different number of clusters and storing the value of the inertia_ property (WCSS) every time.
4. Training the K-Means model on the dataset
Categorize the data using the optimum number of clusters (5) we determined in the last step. k-means++ ensures that you get don't fall into the random initialization trap
5. Generate Statistics from the initialization run with the lowest SSE are available as attributes of kmeans after calling .fit()
6. Creating Output labels for Generating Graph
7. Visualize the clusters

Task to Perform:

1. Implement K-means Clustering Algorithm
2. Visualize your build model in python

Code:**# Importing the libraries**

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
# Importing the dataset
dataset = pd.read_csv('Mall_Customers.csv')
dataset.head()
X = dataset.iloc[:, [3,4]].values
#Using the elbow method to find the optimal number of clusters
from sklearn.cluster import KMeans
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters= i, init='k-means++', random_state= 42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
plt.plot(range(1,11),wcss)
plt.title("Elbow method")
plt.xlabel("No. of clusters")
plt.ylabel("WCSS")
plt.show()
```

```

# Training the K-Means model on the dataset
kmeans = KMeans(n_clusters= 5, init='k-means++',random_state= 42)
kmeans.fit(X)
# Statistics from the initialization run with the lowest SSE are
available as attributes of kmeans after calling .fit()
# The lowest SSE value
kmeans.inertia_
# Final locations of the centroid
kmeans.cluster_centers_
# The number of iterations required to converge
kmeans.n_iter_
#Finally, the cluster assignments are stored as a one-dimensional
NumPy array in kmeans.labels_
kmeans.labels_
#Creating Output labels for Generating Graph
y_kmeans = kmeans.fit_predict(X)
print(y_kmeans)
from sklearn.metrics import accuracy_score
print(accuracy_score(kmeans.labels_,y_kmeans))
#Visualising the clusters
plt.scatter(X[y_kmeans == 0,0],X[y_kmeans == 0,1],s=100, c = 'red',
label ="Cluster 1")
plt.scatter(X[y_kmeans == 1,0],X[y_kmeans == 1,1],s=100, c = 'blue',
label ="Cluster 2")
plt.scatter(X[y_kmeans == 2,0],X[y_kmeans == 2,1],s=100, c =
'lightgreen', label ="Cluster 3")
plt.scatter(X[y_kmeans == 3,0],X[y_kmeans == 3,1],s=100, c = 'black',
label ="Cluster 4")
plt.scatter(X[y_kmeans == 4,0],X[y_kmeans == 4,1],s=100, c =
'magenta', label ="Cluster 5")
plt.scatter(kmeans.cluster_centers_[ :,0],kmeans.cluster_centers_[ :,
1],s = 300, c = 'Yellow',label = 'Centroids')
plt.title("Clusters of Customers",size = 25)
plt.xlabel("Annual Income {k$}")
plt.ylabel("Spending Scores{1 to 100}")
plt.legend()
plt.show()

```

Result:

Optimal number of clusters = _____

Accuracy of Model = _____

Cluster of Test Example = _____

Conclusion:

In this way we successfully implement and visualize K-means Clustering Algorithm in python.

Experiment No. 9

Title: Write a program to implement the Hierarchical Agglomerative Clustering Algorithm for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.

Problem Statement: The main aim of this lab is to demonstrate the how to Hierarchical Agglomerative Clustering Algorithm with Python

Secondary aim of this lab is to understand

- What is Hierarchical Agglomerative Clustering Algorithm?
- How Hierarchical Agglomerative Clustering works?
- What is Dendrograms?
- What is Linkage Criteria?
- How to Fit a Hierarchical Agglomerative Clustering using Scikit-Learn?
- How to Visualize Hierarchical Agglomerative Clustering using Matplotlib?

Software Used: Jupyter Notebook , Python 3.6, Or Google Colab

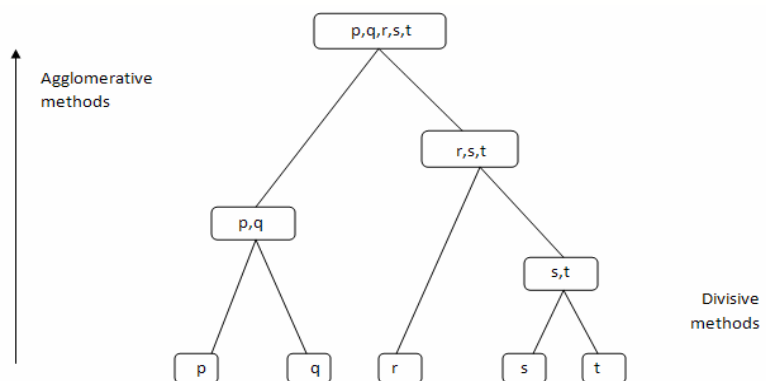
Theory:

Introduction to Hierarchical Clustering

Hierarchical clustering algorithms group similar objects into groups called clusters. There are two types of hierarchical clustering algorithms:

Agglomerative — Bottom up approach. Start with many small clusters and merge them together to create bigger clusters.

Divisive — Top down approach. Start with a single cluster than break it up into smaller clusters.



Some pros and cons of Hierarchical Clustering

Pros

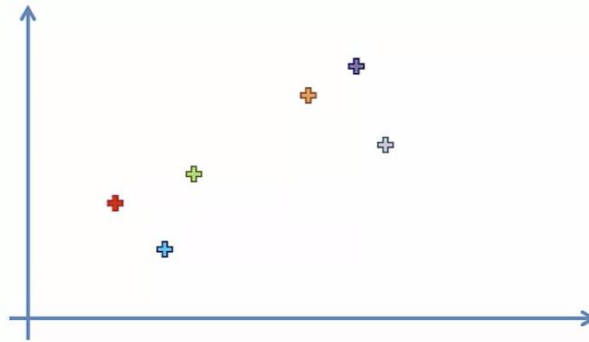
- No assumption of a particular number of clusters (i.e. k-means)
- May correspond to meaningful taxonomies

Cons

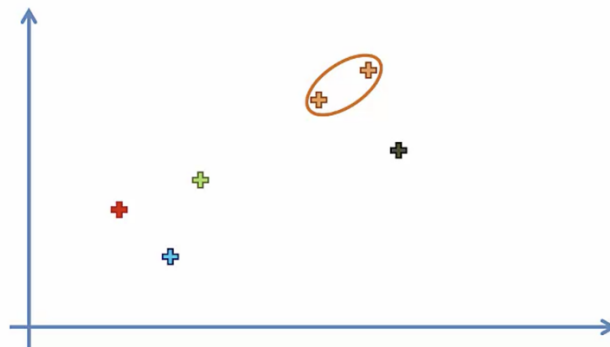
- Once a decision is made to combine two clusters, it can't be undone
- Too slow for large data sets, $O(n^2 \log(n))$

How it works

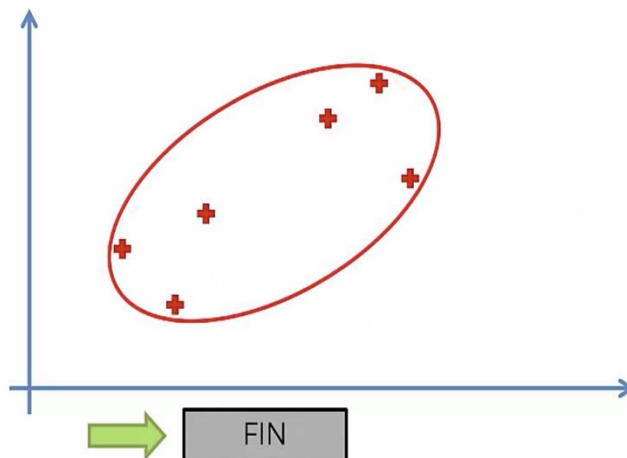
1. Make each data point a cluster



2. Take the two closest clusters and make them one cluster



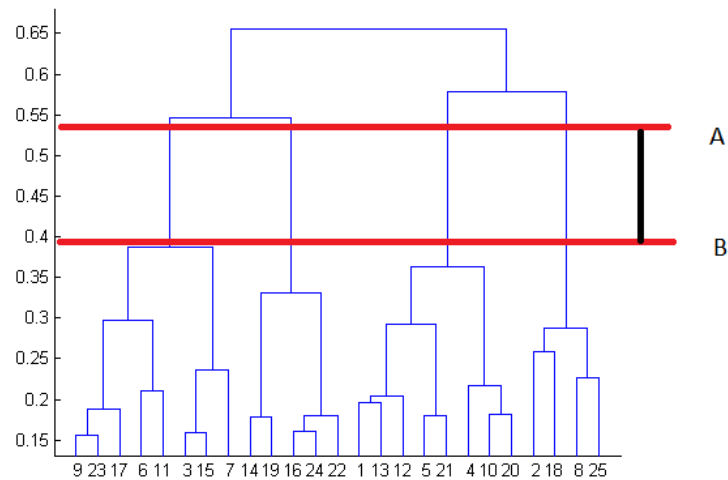
3. Repeat step 2 until there is only one cluster

**Dendrograms**

We can use a dendrogram to visualize the history of groupings and figure out the optimal number of clusters.

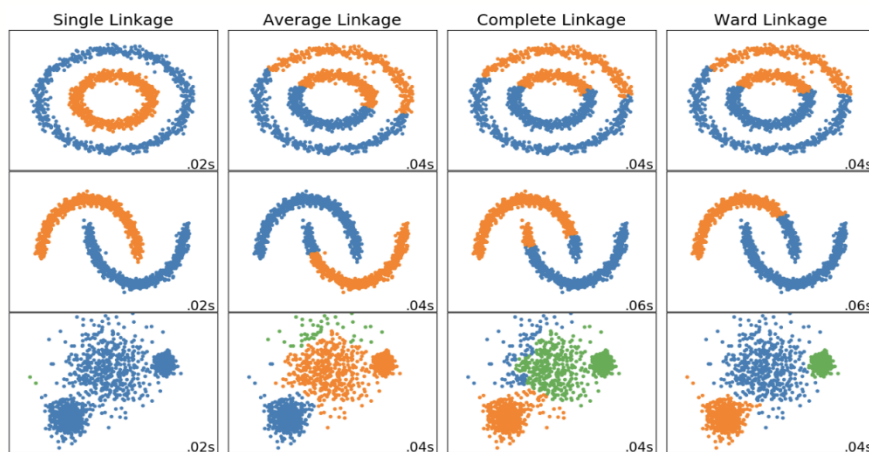
1. **Determine the largest vertical distance that doesn't intersect any of the other clusters**
2. Draw a horizontal line at both extremities
3. The optimal number of clusters is equal to the number of vertical lines going through the horizontal line

For eg., in the below case, best choice for no. of clusters will be 4.



Linkage Criteria

Similar to gradient descent, you can tweak certain parameters to get drastically different results.



The linkage criteria refers to how the distance between clusters is calculated.

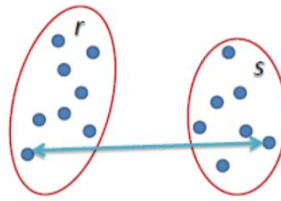
Single Linkage

The distance between two clusters is the shortest distance between two points in each cluster

$$L(r, s) = \min(D(x_{ri}, x_{sj}))$$

Complete Linkage

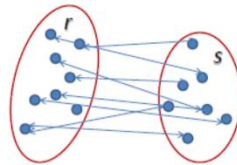
The distance between two clusters is the longest distance between two points in each cluster



$$L(r, s) = \max(D(x_{ri}, x_{sj}))$$

Average Linkage

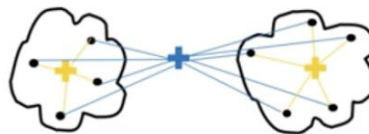
The distance between clusters is the average distance between each point in one cluster to every point in other cluster



$$L(r, s) = \frac{1}{n_r n_s} \sum_{i=1}^{n_r} \sum_{j=1}^{n_s} D(x_{ri}, x_{sj})$$

Ward Linkage

The distance between clusters is the sum of squared differences within all clusters

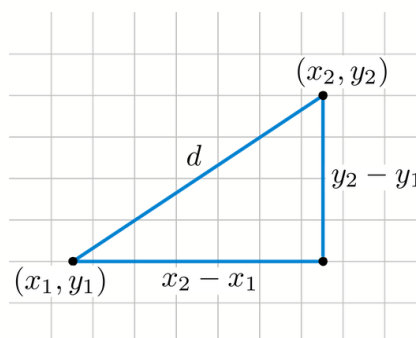


Distance Metric

The method you use to calculate the distance between data points will affect the end result.

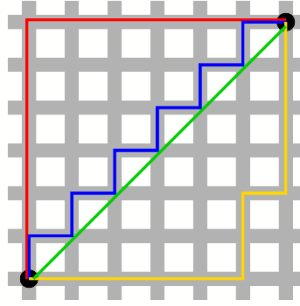
Euclidean Distance

The shortest distance between two points. For example, if $x=(a,b)$ and $y=(c,d)$, the Euclidean distance between x and y is $\sqrt{(a-c)^2+(b-d)^2}$



Manhattan Distance

Imagine you were in the downtown center of a big city and you wanted to get from point A to point B. You wouldn't be able to cut across buildings, rather you'd have to make your way by walking along the various streets. For example, if $x=(a,b)$ and $y=(c,d)$, the Manhattan distance between x and y is $|a-c|+|b-d|$

**Procedure:**

1. Import the required libraries.
2. Import the Dataset.
3. Using the dendrogram to find the optimal number of clusters
 - a. Looking at the dendrogram, the highest vertical distance that doesn't intersect with any clusters is the middle green one. Given that 5 vertical lines cross the threshold, the optimal number of clusters is 5.
4. Training the Hierarchical Clustering model on the dataset
 - a. Create an instance of AgglomerativeClustering using the euclidean distance as the measure of distance between points and ward linkage to calculate the proximity of clusters.
5. Creating Output labels for Generating Graph
6. Visualize the clusters

Task Perfomed:

1. Implement Hierarchical Agglomerative Clustering Algorithm
2. Visualize your build model in python

Code:

```
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
dataset = pd.read_csv("Mall_Customers.csv")
X = dataset.iloc[:, [3,4]].values
print(X[0:5,:])

# Using the dendrogram to find the optimal number of clusters
import scipy.cluster.hierarchy as sch
dendrogram = sch.dendrogram(sch.linkage(X,method = 'ward'))
plt.title("Dendrogram")
plt.xlabel("Customer")
plt.ylabel("Euclidian Distance")

# Training the Hierarchical Clustering model on the dataset
from sklearn.cluster import AgglomerativeClustering
```

```

    hc = AgglomerativeClustering (n_clusters = 5, affinity =
'euclidean', linkage = 'ward')
    y_hc = hc.fit_predict(X)
    print(y_hc)
    # Visualising the clusters
    plt.scatter(X[y_hc == 0, 0], X[y_hc == 0, 1], s = 100, c = 'red',
label = 'Cluster 1')
    plt.scatter(X[y_hc == 1, 0], X[y_hc == 1, 1], s = 100, c = 'blue',
label = 'Cluster 2')
    plt.scatter(X[y_hc == 2, 0], X[y_hc == 2, 1], s = 100, c =
'lightgreen', label = 'Cluster 3')
    plt.scatter(X[y_hc == 3, 0], X[y_hc == 3, 1], s = 100, c = 'black',
label = 'Cluster 4')
    plt.scatter(X[y_hc == 4, 0], X[y_hc == 4, 1], s = 100, c =
'magenta', label = 'Cluster 5')
    plt.title('Clusters of customers')
    plt.xlabel('Annual Income (k$)')
    plt.ylabel('Spending Score (1-100)')
    plt.legend()
    plt.show()

```

Result:

Optimal number of clusters = _____

Accuracy of Model = _____

Cluster of Test Example = _____

Conclusion: In this way we successfully implement and visualize Hierarchical Agglomerative Clustering Algorithm in python.

Experiment No. 10

Title: Create the Model Evaluation for a real life training data set stored as a .CSV file and by computing accuracy for different model proposed the best suitable machine learning algorithm for problem or Solve the any real life problem suggested by Hackerearth site

Problem Statement: The main aim of this lab is to demonstrate the how to Create the Model Evaluation for a real life training data set stored

Secondary aim of this lab is to understand

- What are different evaluation metrics for classification and regression?
- How to evaluate machine learning model?
- How to propose the best fit model using different metrics?
- How to propose the best fit model using graphs?

Software Used: Jupyter Notebook , Python 3.6, Or Google Colab

Dataset Used:

Theory:

Metrics To Evaluate Machine Learning Algorithms in Python :

The metrics that you choose to evaluate your machine learning algorithms are very important.

Choice of metrics influences how the performance of machine learning algorithms is measured and compared. They influence how you weight the importance of different characteristics in the results and your ultimate choice of which algorithm to choose.

In this practical, you will discover how to select and use different machine learning performance metrics in Python with scikit-learn.

There are two parts of evaluation depends upon your dataset and concept:

- A) Evaluating Regression Models
- B) Evaluating Classification Models

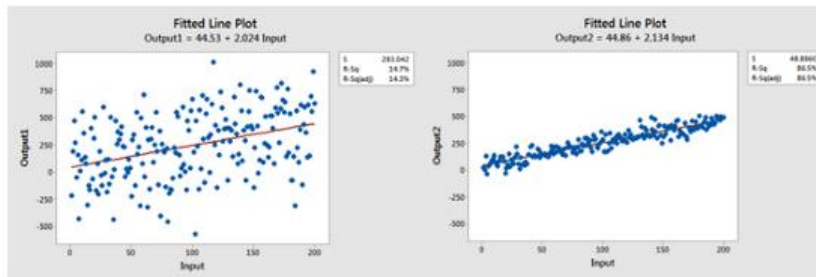
A) Evaluating Regression Models

1. R-Squared

R Squared is a measurement that tells you to what extent the proportion of variance in the dependent variable is explained by the variance in the independent variables. In simpler terms, while the coefficients estimate trends, R-squared represents the scatter around the line of best fit.

For example, if the R^2 is 0.80, then 80% of the variation can be explained by the model's inputs. If the R^2 is 1.0 or 100%, that means that all movements of the dependent variable can be entirely explained by the movements of the independent variables.

To show a visual example, despite having the same line of best fit, the R^2 on the right is much higher than the one on the left.



The equation for R^2 is as follows:

$$R^2 = 1 - \frac{\text{Explained Variation}}{\text{Total Variation}}$$

The Explained Variation is equal to the sum of squared residuals while the total variation is equal to the total sum of squared.

$$SS_{\text{residual}} = \sum_{i=0}^n (y_i - \hat{y}_i)^2$$

$$SS_{\text{total}} = \sum_{i=0}^n (y_i - y_{\text{avg}})^2$$

Now that you understand what R^2 is, the code is very straight forward!

```
from sklearn.metrics import r2_score
sklearn.metrics.r2_score(y_true, y_pred)
```

2. Adjusted R-Squared

Every additional independent variable added to a model always increases the R^2 value — therefore, a model with several independent variables may seem to be a better fit even if it isn't. This is where Adjusted R^2 comes in. The adjusted R^2 compensates for each additional independent variable and only increases if each given variable improves the model above what is possible by probability.

There are a couple of ways to find the adjusted R^2 with Python:

Option 1: Manual Calculation

```
# n = number of sample size
# p = number of independent variables
Adj_r2 = 1 - (1 - R2) * (n - 1) / (n - p - 1)
```

Option 2: statsmodel.api

```
import statsmodels.api as sm
from statsmodels.sandbox.regression.predstd import wls_prediction_std
model1=sm.OLS(y_train,x_train)
result=model1.fit()
print(result.summary())
```

3. Mean Absolute Error (MAE)

The absolute error is the difference between the predicted values and the actual values. Thus, the mean absolute error is the average of the absolute error.

$$\text{Mean Absolute Error} = \frac{1}{N} \sum_{i=0}^n |y_i - \hat{y}_i|$$

By importing `mean_absolute_error` from `sklearn.metrics`, you can easily compute the MAE of your model.

```
from sklearn.metrics import mean_absolute_error
mean_absolute_error(y_true, y_pred)
```

4. Mean Squared Error (MSE)

The mean squared error or MSE is similar to the MAE, except you take the average of the squared differences between the predicted values and the actual values.

Because the differences are squared, larger errors are weighted more highly, and so this should be used over the MAE when you want to minimize large errors. Below is the equation for MSE, as well as the code.

$$\text{Mean Squared Error} = \frac{1}{N} \sum_{i=0}^n (y_i - \hat{y}_i)^2$$

```
from sklearn.metrics import mean_squared_error
mean_squared_error(y_true, y_pred)
```

B) Evaluating Classification Models

1. Confusion Matrix and related metrics

A confusion matrix, also known as an error matrix, is a performance measurement for assessing classification models. Below is an example of a two-class confusion matrix.

	Predicted: No	Predicted: Yes
Actual: No	True Negative 50	False Positive 10
Actual: Yes	False Negative 5	True Positive 100

Within the confusion matrix, there are some terms that you need to know, which can then be used to calculate various metrics:

True Positive: Outcome where the model correctly predicts the positive class.

True Negative: Outcome where the model correctly predicts the negative class.

False Positive (Type 1 Error): Outcome where the model incorrectly predicts the positive class.

False Negative (Type 2 Error): Outcome where the model incorrectly predicts the negative class.

Now that you know these terms, here are a number of metrics that you can calculate:

Accuracy: equal to the fraction of predictions that a model got right.

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

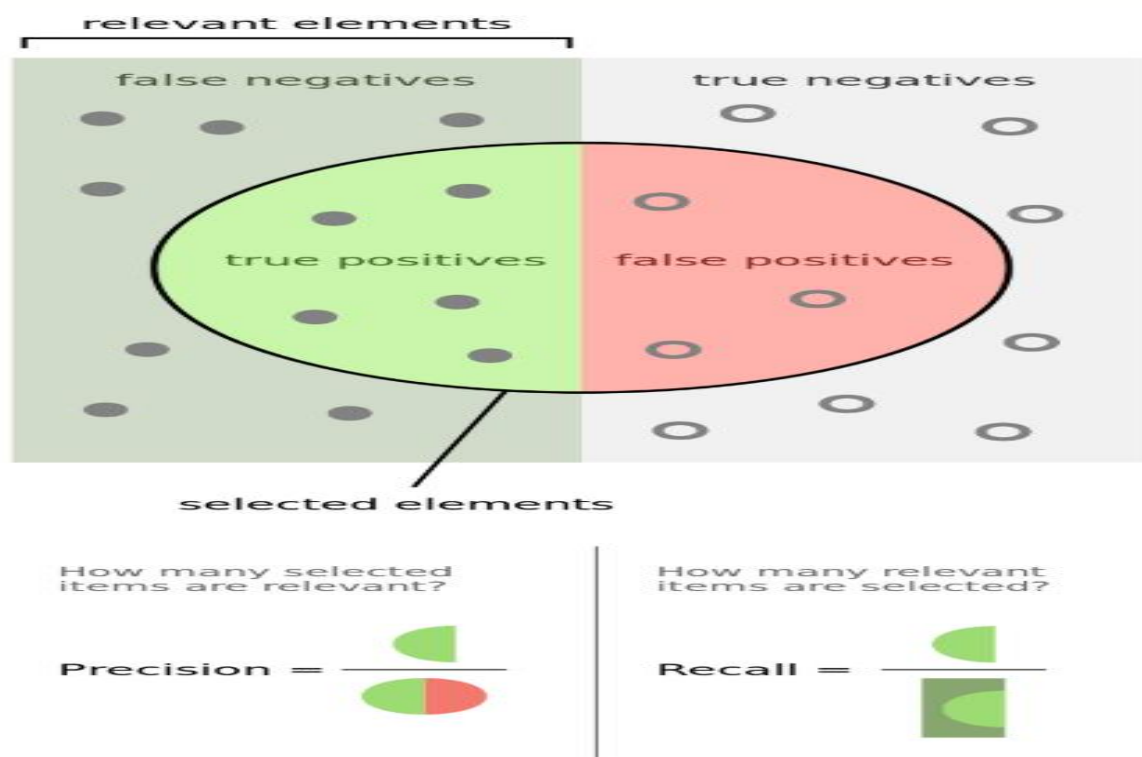
Recall: attempts to answer “What proportion of actual positives was identified correctly?”

$$\text{Recall} = \frac{TP}{TP + FN}$$

Precision: attempts to answer “What proportion of positive identifications was actually correct?”

$$\text{Precision} = \frac{TP}{TP + FP}$$

To really hit it home, the diagram below is a great way to remember the difference between precision and recall.



Code for confusion matrix and related metrics are below:

```
# Confusion Matrix
from sklearn.metrics import confusion_matrix
confusion_matrix(y_true, y_pred)

# Accuracy
from sklearn.metrics import accuracy_score
```



```
accuracy_score(y_true, y_pred)

# Recall
from sklearn.metrics import recall_score
recall_score(y_true, y_pred, average=None)

# Precision
from sklearn.metrics import precision_score
precision_score(y_true, y_pred, average=None)
```

2. F1 Score

$$F1 = \frac{2}{\frac{1}{precision} + \frac{1}{recall}} = \frac{2 * (precision * recall)}{precision + recall}$$

The F1 score is a measure of a test's accuracy — it is the harmonic mean of precision and recall. It can have a maximum score of 1 (perfect precision and recall) and a minimum of 0. Overall, it is a measure of the preciseness and robustness of your model.

There are three ways you can calculate the F1 score in Python:

```
# Method 1: sklearn
from sklearn.metrics import f1_score
f1_score(y_true, y_pred, average=None)

# Method 2: Manual Calculation
F1 = 2 * (precision * recall) / (precision + recall)

# Method 3: BONUS - classification report
from sklearn.metrics import classification_report
print(classification_report(y_true, y_pred, target_names=target_names))
```

3. AUC-ROC Curve

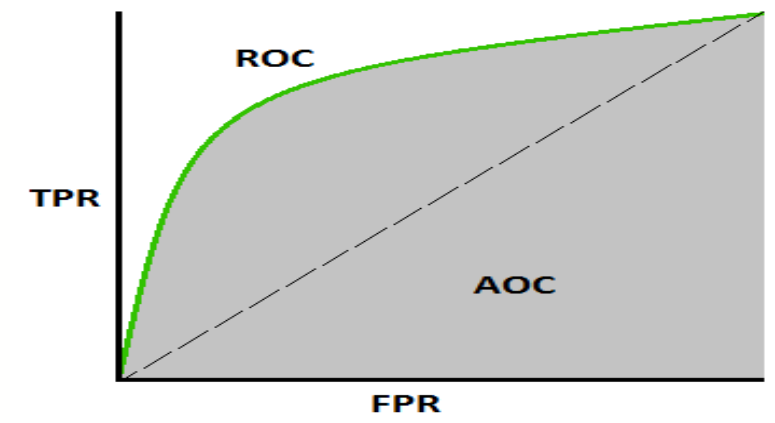
The AUC-ROC Curve is a performance measurement for classification problems that tells us how much a model is capable of distinguishing between classes. A higher AUC means that a model is more accurate.

What is the AUC - ROC Curve?

AUC - ROC curve is a performance measurement for the classification problems at various threshold settings. ROC is a probability curve and AUC represents the degree or measure of separability. It tells how much the model is capable of distinguishing between classes. Higher the AUC, the better the model is at predicting 0 classes as 0 and 1 classes as 1. By analogy, the Higher the AUC, the better the model is at distinguishing between patients with the disease and no disease.

The ROC curve is plotted with TPR against the FPR where TPR is on the y-axis and FPR is on the x-axis.

If you want to learn more about it, Sarang does a great job explaining it here.
To calculate the AUC-ROC score, you can replicate the code below:



Defining terms used in AUC and ROC Curve.

TPR (True Positive Rate) / Recall / Sensitivity

$$\text{TPR / Recall / Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Specificity

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

FPR

$$\begin{aligned} \text{FPR} &= 1 - \text{Specificity} \\ &= \frac{\text{FP}}{\text{TN} + \text{FP}} \end{aligned}$$

How to speculate about the performance of the model?

An excellent model has AUC near to the 1 which means it has a good measure of separability. A poor model has an AUC near 0 which means it has the worst measure of separability. In fact, it means it is reciprocating the result. It is predicting 0s as 1s and 1s as 0s. And when AUC is 0.5, it means the model has no class separation capacity whatsoever.

To calculate the AUC-ROC score, you can replicate the code below:

```
import numpy as np
from sklearn.metrics import roc_auc_score
y_true = np.array([0, 0, 1, 1])
y_scores = np.array([0.1, 0.4, 0.35, 0.8])
roc_auc_score(y_true, y_scores)
0.75
```

4. jaccard_score

The Jaccard similarity index measures the similarity between two sets of data. It can range from 0 to 1. The higher the number, the more similar the two sets of data.

The Jaccard similarity index is calculated as:

Jaccard Similarity = (number of observations in both sets) / (number in either set)

Or, written in notation form:

$$J(A, B) = |A \cap B| / |A \cup B|$$

This tutorial explains how to calculate Jaccard Similarity for two sets of data in Python.

Jaccard similarity coefficient score

The Jaccard index [1], or Jaccard similarity coefficient, defined as the size of the intersection divided by the size of the union of two label sets, is used to compare set of predicted labels for a sample to the corresponding set of labels in ``y\_true``.

Example: Jaccard Similarity in Python

```
from sklearn.metrics import jaccard_score
y_true = np.array([[0, 1, 1],
...               [1, 1, 0]])
y_pred = np.array([[1, 1, 1],
...               [1, 0, 0]])
```

In the binary case:

```
jaccard_score(y_true[0], y_pred[0]) # doctest: +ELLIPSIS
0.6666...
```

In the multilabel case:

```
jaccard_score(y_true, y_pred, average='samples')
0.5833...
```

```
jaccard_score(y_true, y_pred, average='macro')
0.6666...
```

```
jaccard_score(y_true, y_pred, average=None)
array([0.5, 0.5, 1. ])
```

5. Log_loss

Log loss, aka logistic loss or cross-entropy loss.

This is the loss function used in (multinomial) logistic regression and extensions of it such as neural networks, defined as the negative log-likelihood of a logistic model that returns `y_pred` probabilities for its training data `y_true`. The log loss is only defined for two or more labels. For a single sample with true label $y \in \{0,1\}$ and a probability estimate $p = \Pr(y=1)$, the log loss is:

$$L_{\log}(y, p) = -(y \log(p) + (1 - y) \log(1 - p))$$

Examples

```
from sklearn.metrics import log_loss
```

```
log_loss(["spam", "ham", "ham", "spam"],
[[.1, .9], [.9, .1], [.8, .2], [.35, .65]])

0.21616...
```

Procedure:

1. Import Libraries
2. Import the Dataset.
3. Do the Exploratory data analysis
4. Do the preprocessing of data
5. Apply the classification/regression) models (min 3 and max 5) do you want wish to train your data
6. Do the model evaluation applicable to your model
7. Create the evaluation table
8. Visualize the evaluation model
9. Do the cross validation and proposed the best model for your dataset
10. If you are using classification build the ROC curve and discuss it
11. If you are using Regression model build the residual chart.
12. Conclude with best model for your dataset.

Task Performed: Select Dataset and perform evaluation of model using atleast four algorithm of machine Learning

Result and Discussion

The best model for our dataset is

With following metrics of evaluation

Comment What ROC Curve/ Residual chart suggest us

Conclusion: In this way we successfully perform model evaluation on downloaded dataset.